Computer Science and Engineering

Software Engineering II

2025 – 2026

Design Document

Best Bike Paths

Authors:
Wojciech Maciej Bukala (11152502)
Giacomo Rigo (11106172)
Mattia Zonzin (10849943)

Professor:
Prof. Elisabetta Di Nitto

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Scope . . . 3
1.3 Definitions, Acronyms, Abbreviations . . . 3
1.3.1 Definitions . . . 3
1.3.2 Acronyms . . . 4
1.3.3 Abbreviations . . . 5
1.4 Reference Documents . . . 5
1.5 Revision history . . . 5
1.6 Document Structure . . . 5

2 Architectural Design **7**
2.1 Overview . . . 7
2.2 Component view . . . 9
2.2.1 Presentation components . . . 9
2.2.2 Application components . . . 10
2.2.3 Data components . . . 11
2.2.4 External component . . . 11
2.3 Deployment View . . . 12
2.4 Runtime View . . . 14
2.4.1 UC1 – Search Paths . . . 14
2.4.2 UC2 – View Path Details . . . 15
2.4.3 UC3 – Admin Login . . . 16
2.4.4 UC4 – Configure Data Merging Policy . . . 17
2.4.5 UC5 – Admin Edit Path . . . 18
2.4.6 UC6 – Block User . . . 19
2.4.7 UC7 – User Registration . . . 20
2.4.8 UC8 – User Login . . . 21
2.4.9 UC9 – Record Trip . . . 22
2.4.10 UC10 – Manual Path Insertion . . . 23
2.4.11 UC11 – Store Trip . . . 24
2.4.12 UC12 – Publish Path . . . 25
2.4.13 UC13 – View Trip History . . . 26
2.4.14 UC14 – Validate Automatically Created Path . . . 28
2.4.15 UC15 – Publish Trip . . . 30
2.4.16 UC16 – Manage Path (User Side) . . . 31
2.4.17 UC16 – Manage Path (Admin Side) . . . 32
2.5 Component Interfaces . . . 33
2.5.1 API Interfaces . . . 34

2.5.2 DBMS Interfaces 36
2.6 Selected Architectural Styles and Patterns 37
2.6.1 Presentation Tier 38
2.6.2 Application Tier - Web Layer 38
2.6.3 Application Tier - Backend Layer 39
2.6.4 Data Tier 39
2.7 Other Design Decisions 39
2.7.1 External Map Service 40
2.7.2 External Weather Service 40
2.7.3 Device Sensor Interfaces 40
2.7.4 Communication Interfaces and Interaction Model 41

3 User Interface Design 42
3.1 Home View 43
3.2 Record Page 44
3.3 History Page 45
3.4 Manual path insertion 46
3.5 Publication Page 47
3.6 Settings Page 48

4 Requirements Traceability 49
4.1 User Registration and Account Management 49
4.2 Trip Recording, Statistics and Publication 49
4.3 Path Contribution and Validation 50
4.4 Path Search and Routing 50
4.5 Data Aggregation and Governance 51

5 Implementation, Integration & Test Plan 52
5.1 Overview and Implementation Plan 52
5.2 Features Identification 53
5.3 Component Integration and Testing 54
5.4 System testing 61
5.5 Additional Specifications on Testing 62

6 Effort spent 64

7 References 65

1 Introduction

1.1 Purpose

Cycling is an increasingly popular mode of transport and recreation, recognized for its environmental and health benefits. As communities of riders grow, there is a need for digital solutions that facilitate data sharing and improve route safety. The Best Bike Paths platform is designed to address these needs by providing a data-driven environment for cyclists.
The purpose of this Design Document is to provide a detailed technical specification and architectural overview of the Best Bike Paths system. This document serves as a technical reference for the development team, describing the system's component structure, API interfaces, deployment views, and the plans for integration and testing. It outlines the key design decisions made to fulfill the platform's functional and non-functional requirements, establishing a foundation for its secure and scalable implementation.

1.2 Scope

The Best Bike Paths (BBP) platform is a digital ecosystem designed for cycling route management and community data exchange. It allows users to access validated bike paths, record trips manually or automatically via GPS and sensors, and integrate external meteorological data to provide a comprehensive cycling experience.
This document defines the technical scope and design specifications necessary to realize this platform. It outlines the system's 4-tier architecture, the detailed microservices design, and the specification of both internal and external interfaces. Additionally, the scope covers user interface design principles, requirements traceability, and the comprehensive plan for implementation, integration, and testing. By establishing these boundaries, the document provides a technical roadmap for the successful development and deployment of the Best Bike Paths system.

1.3 Definitions, Acronyms, Abbreviations

1.3.1 Definitions

The following terms are used throughout this document with the meanings specified below.

- **User**: Any person interacting with the BBP platform. Users can be either registered or unregistered.
- **General User**: Person interacting with the BBP platform that is not logged in.

- **Registered User**: Person interacting with the BPP platform that we are sure that is logged in.
- **BBP Administrator**: A user representing the Bikers' Association, responsible for configuring system policies and moderating community-provided content.
- **Trip/Ride**: A cycling activity performed by a registered user.
- **Bike Path/Cycling Route**: A route suitable for cycling, consisting of either dedicated cycling infrastructure or road segments with limited traffic and speed conditions compatible with bicycles.
- **Automatic Recording**: A mode in which the system collects trip data using a user's mobile device sensors.
- **Manual Path Insertion**: The process by which a registered user manually inputs details about a cycling route.
- **Route Ranking**: The mechanism through which available cycling routes are presented to users in an ordered manner based on factors such as surface quality, road safety, community feedback, and route efficiency.
- **Obstacle**: Any road condition or irregularity that may affect cycling safety or comfort, such as potholes.

1.3.2 Acronyms

- **DD**: Design Document
- **BBP**: Best Bike Paths
- **GPS**: Global Positioning System
- **API**: Application Programming Interface
- **DBMS**: Database Management System
- **HTTP**: Hypertext Transfer Protocol
- **HTTPS**: Hypertext Transfer Protocol Secure
- **REST**: Representational state transfer
- **JSON**: JavaScript Object Notation

1.3.3 Abbreviations

- **UC**: Use Case
- **R**: Requirement
- **UX**: User Experience

1.4 Reference Documents

The content of this document, along with the related analysis and designs, is based on the following sources:

- The official specification of the 2024/2025 Requirements Engineering and Design Project for the Software Engineering II course.
- The lecture slides and learning materials available on the WeBeep platform for the Software Engineering II course.

1.5 Revision history

- 05-01-2026: Version 1.0

1.6 Document Structure

1. **Introduction**
   Provides a brief description of the Best Bike Paths (BBP) platform, outlining its purpose and defining the scope of the product.
2. **Architectural Design**
   Describes the system architecture both at a high level and at the level of individual components. Additionally, it explains the interactions between the architectural layers.
3. **User Interface Design**
   Presents the appearance and behavior of the platform from the user's perspective.
4. **Implementation, Integration, and Test Plan**
   Focuses on implementation strategies, integration details, and testing procedures to help developers fully understand the intentions of the design team.

5. **Effort Spent**
   Summarizes the contribution and time dedicated by each member of the group to the creation of this document.

6. **References**
   Lists all external sources, documents, and tools used in the preparation and writing of this Design Document (DD).

2 Architectural Design

2.1 Overview

The BBP system adopts a **4-Tier** architectural model aimed at ensuring separation of concerns, scalability, and maintainability. The architecture is composed of a Presentation Tier, an Application Tier split into two distinct layers (Web Layer and Backend Layer), and a Data Tier. Each tier encapsulates a well-defined set of responsibilities and interacts with adjacent tiers through clearly defined interfaces.

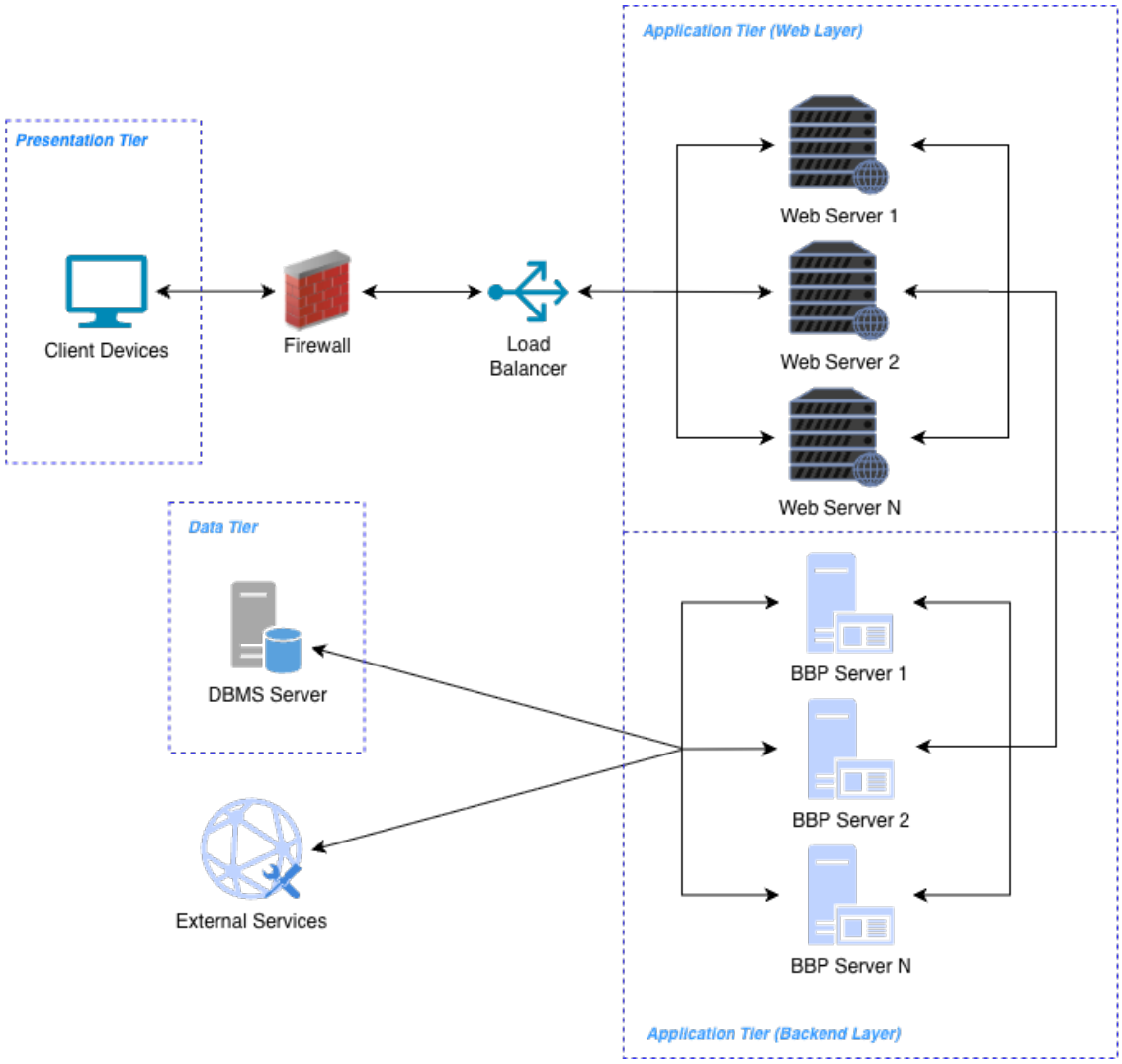


Figure 1: 4-Tier architecture overview.

Presentation Tier

The Presentation Tier represents the user-facing layer of the system and is responsible for interaction with end users. It includes web and mobile client applications that provide the graphical user interface and handle user input. This tier focuses exclusively on presentation logic and user experience, delegating all application processing and data management to the underlying application layers. Communication with the backend is performed through standardized and secure interfaces, ensuring that the Presentation Tier remains independent from internal implementation details.

Application Tier

The Application Tier implements the core functionality of the system and is logically divided into two complementary layers:

- **Web Layer**: this layer acts as the entry point to the application logic. It is responsible for handling incoming requests, managing communication protocols, and routing requests towards the backend services. By isolating these concerns, the Web Layer shields the backend from direct client interaction and supports scalability and traffic management.
- **Backend Layer**: this layer contains the core business logic of the system. It processes application requests, enforces business rules, manages authentication and authorization, and coordinates access to persistent data and external services. The separation between Web and Backend layers improves modularity and allows each layer to evolve independently.

Data Tier

The Data Tier is responsible for persistent data storage and management. It hosts the database management system that stores all long-term system data. From an architectural perspective, the DBMS may be logically partitioned into multiple database partitions to support scalability, fault tolerance, and data redundancy. This tier is accessed exclusively by the Backend Application Layer, ensuring controlled and consistent data access.

2.2 Component view

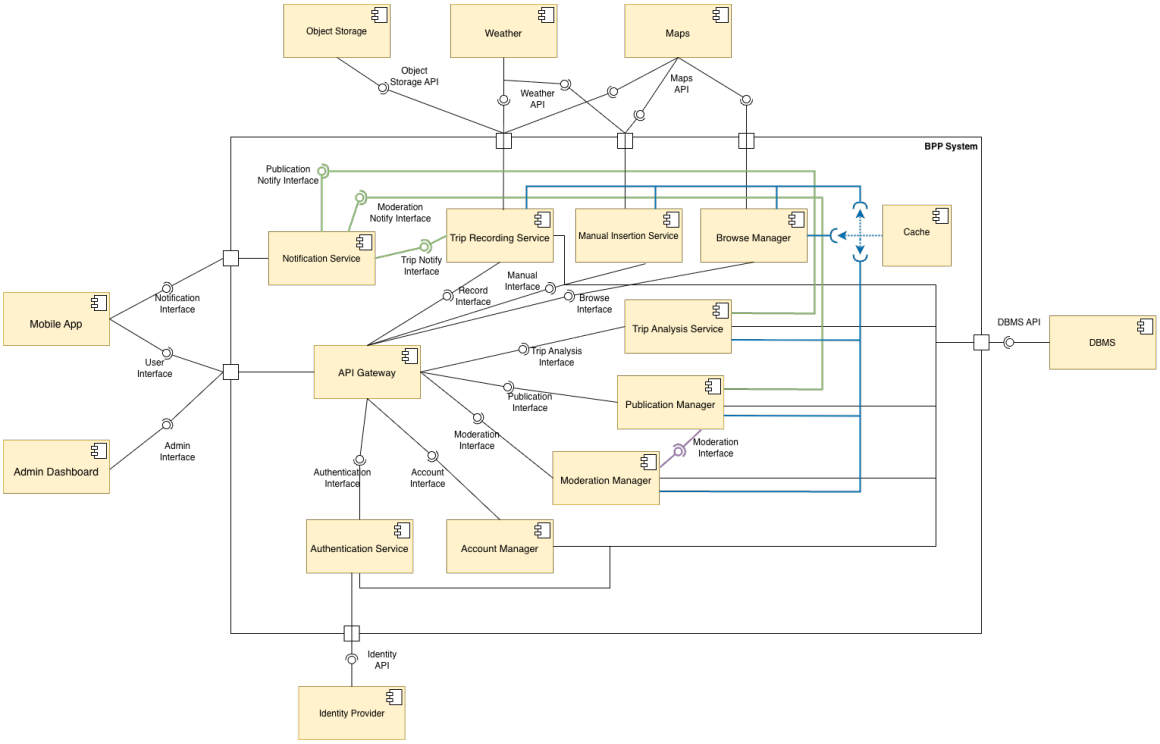


Figure 2: 3-Tier architecture overview.

The Component View shows the decomposition of the Best Bike Paths system into nodular units of implementation. This diagram details the internal composition of each tier from 4-tier architecture, especially focusing on the breakdown of the Application Tier into microservices.

2.2.1 Presentation components

- **Mobile App**
  Mobile application allowing the user to interact with the system. Designed according to the **Thick Client** philosophy. It is responsible for sampling telemetry data directly from device sensors, rendering map tiles, and buffering trip data locally to support offline operations before synchronization.

- **Admin Dashboard**
  A web-based application designed strictly for system administrators. It provides tools to control user's suspicious behaviors and errors, as well as general condition of the system.

2.2.2 Application components

- **API Gateway**
  A dispatcher of requests for services in the application tier. Every request coming from user and administrator is first handled by this component and redirected to the suitable service. In addition, it manages the translation of the protocol and preliminary security checks.

- **Notification Service**
  A dedicated service for managing outbound communication. It can be triggered by a Trip Recording Service to notify about recording state, by a Moderation Manager to inform the user about administrator's actions, or by Publication Manager to update the user on his public paths.

- **Authentication Service**
  Manages user identification and access control by issuing and validating security tokens. Ensures that only authorized requests reach the system resources.

- **Account Manager**
  Handle user profile life cycle. It allows users to review and edit their account data, manages social connections, and track user's trips.

- **Trip Recording Service**
  A service responsible for ingestion of trip data uploaded by the Mobile App. Validates automatically recorded trips.

- **Manual Insertion Service**
  Component responsible for processing and validating data of manually inserted path.

- **Browse Manager**
  A read-optimized service dedicated to serving map data and existing paths. It is coupled with the caching layer to ensure low-latency responses.

- **Trip Analysis Service**
  Service providing computational engine, for analyzing trips.

- **Publication Manager**
  Service for managing the publication process. It ensures that the published trip is valid and notify the user if any problem occurs.

- **Moderation Manager**
  Service to organize merging of obstacles reports and rating paths. Merging policy can be changed by administrator through this component.

- **Cache**
  Component that works as an in-memory data store that is used to temporarily hold accessed data. It helps reduce the load on the DBMS and improves the response time of the system.

2.2.3 Data components

- **DBMS**
  Component responsible for interaction between components from Application Tier and Database. The provided API ensures methods for storing, retrieving and updating data in the system.

- **Object Storage**
  API provided by this component let other components store files such as, for example, images, or other that won't be stored in the database.

2.2.4 External component

- **Weather**
  An external API used to retrieve weather conditions, used by Trip Recording Service to enrich trip data.

- **Maps**
  Component used by the system to fetch base map tiles for rendering, and to perform geocoding.

- **Identity Provider**
  External authentication service allowing users to register and sign in using existing credentials (e.g. Google OAuth, Facebook Login), simplifying registration process.

2.3 Deployment View

This section describes the deployment architecture of the BBP system by detailing the devices involved in each architectural tier and their responsibilities.

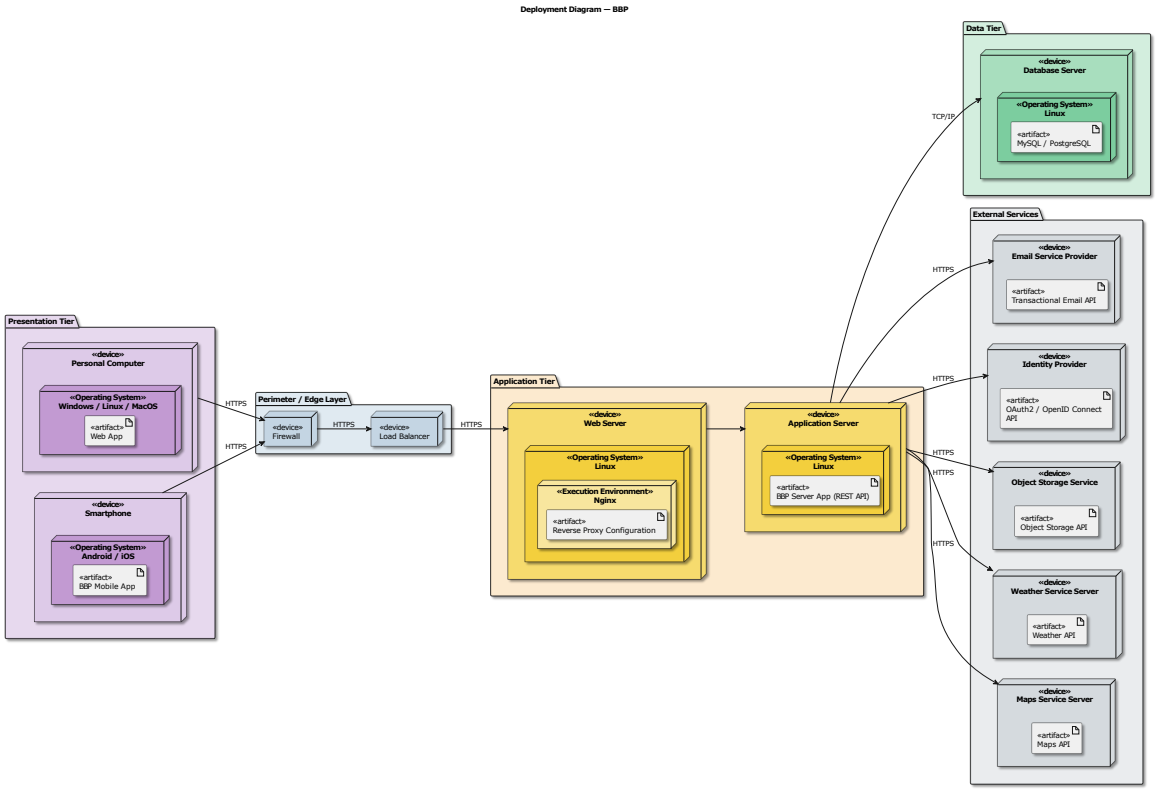


Figure 3: Deployment diagram of the BBP system

Presentation Tier

The Presentation Tier represents the client-side devices from which users access the system at runtime:

- **Personal Computer**: a client device running a standard operating system (Windows, Linux, or macOS) and a web browser. It executes the BBP Web Application, which provides access to the system functionalities through a graphical user interface. Communication with the system is performed exclusively over secure HTTPS connections.
- **Smartphone**: a mobile client device running Android or iOS. It hosts the BBP Mobile Application, offering functionality equivalent to the web application and enabling the use of mobile-specific features such as GPS and onboard sensors. All interactions with the backend occur over HTTPS.

Perimeter / Edge Layer

The Perimeter / Edge Layer includes the infrastructure components deployed at the boundary of the system and responsible for traffic control and security enforcement:

- **Firewall**: filters incoming network traffic and enforces access control policies, preventing unauthorized access to the internal deployment nodes.
- **Load Balancer**: distributes incoming client requests across multiple Web Server instances, supporting scalability, load distribution, and high availability.

Application Tier (Web Layer)

The Web Layer of the Application Tier hosts the components responsible for handling incoming requests:

- **Web Server**: a Linux-based server running Nginx as execution environment. It operates as a reverse proxy, terminating HTTPS connections, performing request routing, and forwarding validated requests to the backend application servers. Multiple equivalent Web Server instances can be deployed for fault tolerance and scalability, although they are logically represented as a single node in the deployment diagram.

Application Tier (Backend Layer)

The Backend Layer hosts the core application services:

- **Application Server**: a Linux-based server running the BBP Server Application. It exposes the system RESTful API, implements business logic, manages authentication and authorization, and coordinates interactions with the Data Tier and with External Services. The backend layer may be replicated across multiple instances to improve availability and performance.

Data Tier

The Data Tier is responsible for persistent data storage:

- **Database Server**: a dedicated Linux-based server running a relational database management system (e.g., MySQL or PostgreSQL). The DBMS may be logically partitioned to improve scalability, redundancy, and fault tolerance. Access to the database is restricted to the Backend Application Layer through a protected internal network.

External Services

External Services represent third-party systems accessed at runtime by the backend application servers:

- **Email Service Provider**: sends transactional emails such as account verification, notifications, and password recovery messages via a secure HTTPS-based API.

- **Identity Provider**: supports user authentication and federation through OAuth 2.0 and OpenID Connect protocols, while authorization is managed by the backend application.
- **Object Storage Service**: stores large binary objects (e.g., raw trip recordings) that are not persisted in the relational database and is accessed via a dedicated storage API.
- **Weather Service Server**: provides weather-related data through a public HTTPS API, used to enrich application features.
- **Maps Service Server**: offers geographic, routing, and geocoding functionalities through an HTTPS API, supporting navigation and spatial processing features.

2.4 Runtime View

2.4.1 UC1 – Search Paths

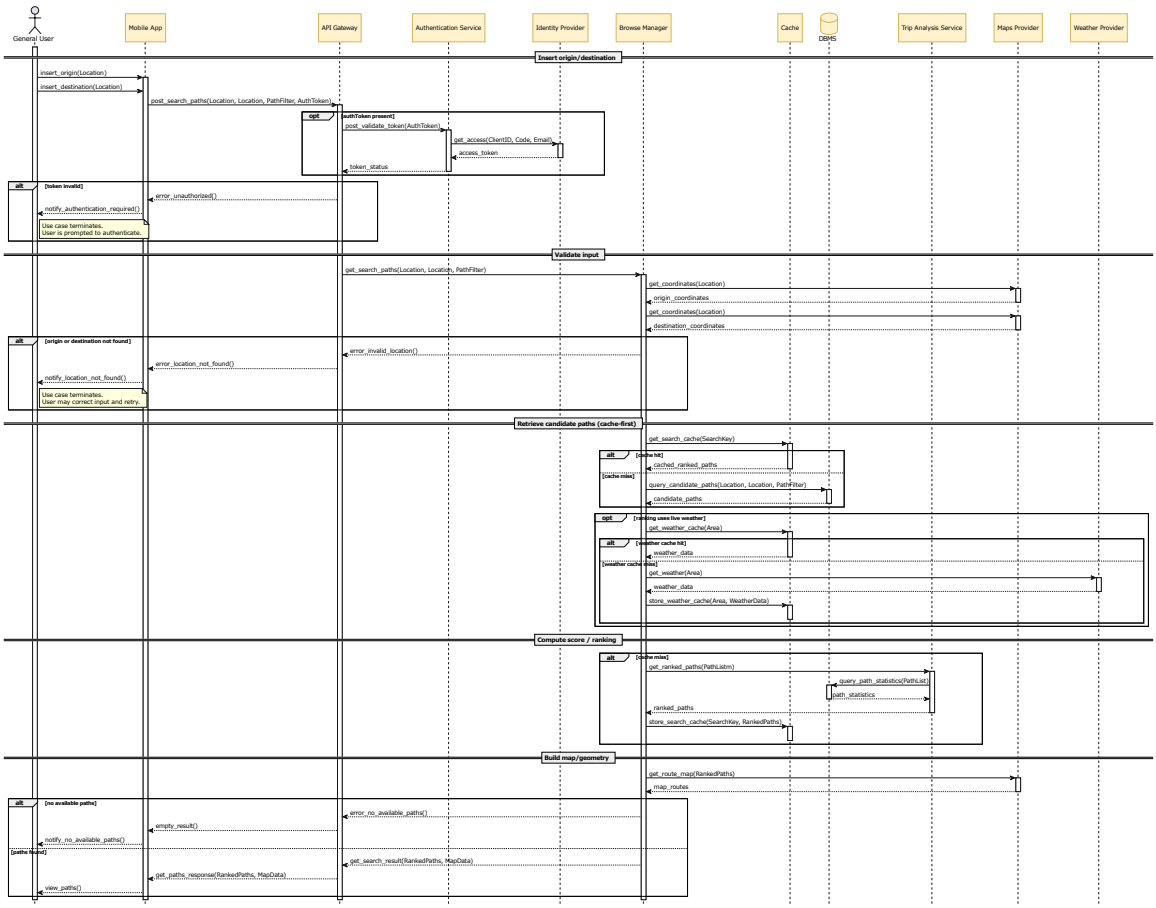


Figure 4: UC1 – Search Paths

The *Search Paths* use case describes the process through which a general user searches for available paths between an origin and a destination. The interaction starts from the **Mo-**

bile App, which collects the input data and forwards the request to the **API Gateway**. If an authentication token is provided, the Gateway delegates its validation to the **Authentication Service**, which relies on the external **Identity Provider**. In case of invalid credentials, the use case is terminated and the user is notified.
Once access is granted, the **Browse Manager** coordinates the search operation. Origin and destination are first validated through the external **Maps Provider**, ensuring that both locations can be resolved into geographic coordinates. If the validation fails, the process is interrupted and an error is returned to the user.
After input validation, the Browse Manager applies a *cache-first* strategy to retrieve previously computed search results. If no cached data is available, candidate paths are retrieved from the **DBMS**. When required, real-time weather information is obtained from the **Weather Provider** and temporarily stored in the cache. The candidate paths are then passed to the **Trip Analysis Service**, which computes the ranking based on historical and statistical data stored in the DBMS. Finally, the path geometry is generated through the Maps Provider and the ranked results are returned to the user for visualization, or an empty result is reported if no valid paths are found.

2.4.2 UC2 – View Path Details

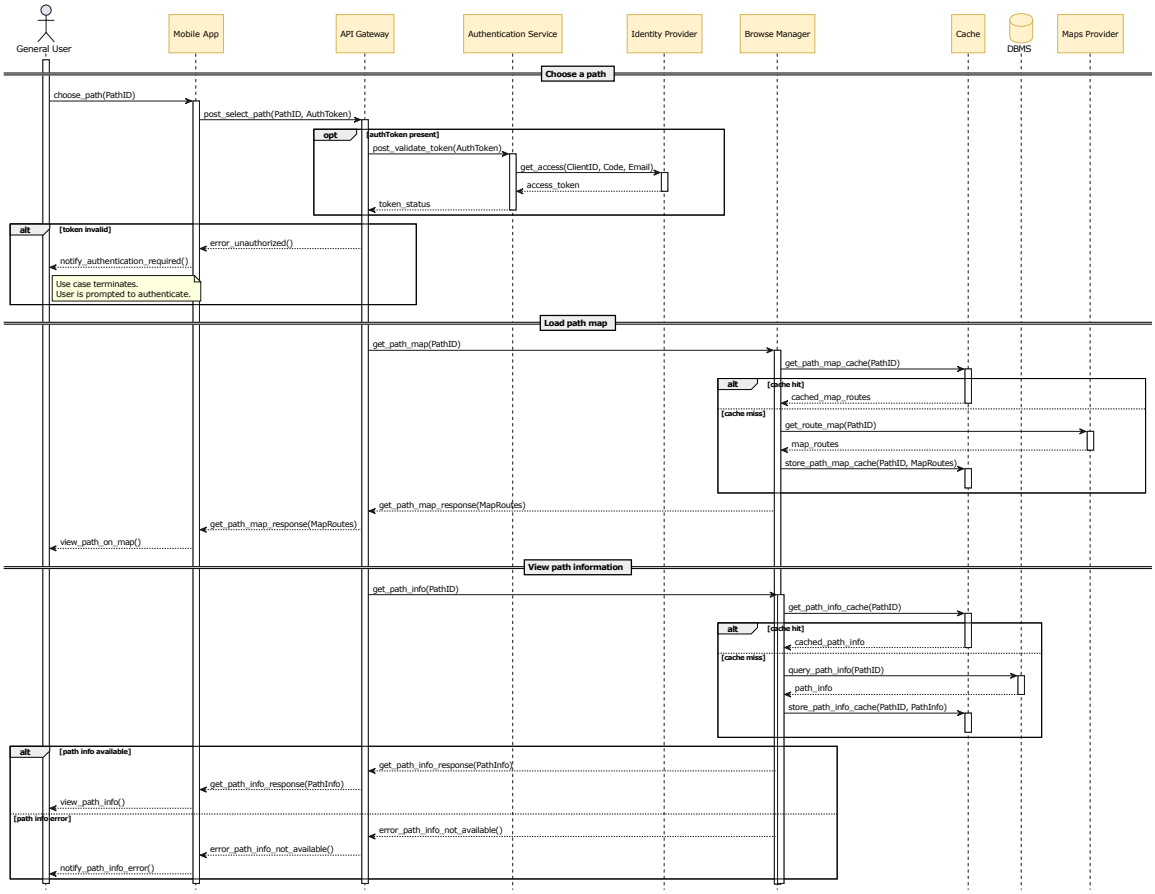


Figure 5: UC2 – View Path Details

The *View Path Details* use case describes the process through which a general user selects a previously identified path and visualizes its details and geographic representation. The interaction starts from the **Mobile App**, which sends the selected *PathID* to the **API Gateway**. If an authentication token is provided, the Gateway validates it through the **Authentication Service** and the external **Identity Provider**; in case of invalid credentials, the use case is terminated and the user is prompted to authenticate.
Once access is granted, the **Browse Manager** retrieves the path map by first querying the **Cache**. In case of a cache miss, the geometric representation of the path is obtained from the external **Maps Provider** and stored in the cache for future requests. The map data is then returned to the user for visualization.
Subsequently, the Browse Manager retrieves additional information associated with the selected path, such as metadata and descriptive attributes. This information is again obtained using a cache-first strategy, falling back to the **DBMS** if no cached data is available. If the path information is successfully retrieved, it is returned to the user; otherwise, an error notification is issued. This approach ensures efficient data retrieval while maintaining consistency between cached and persistent data sources.

2.4.3 UC3 – Admin Login

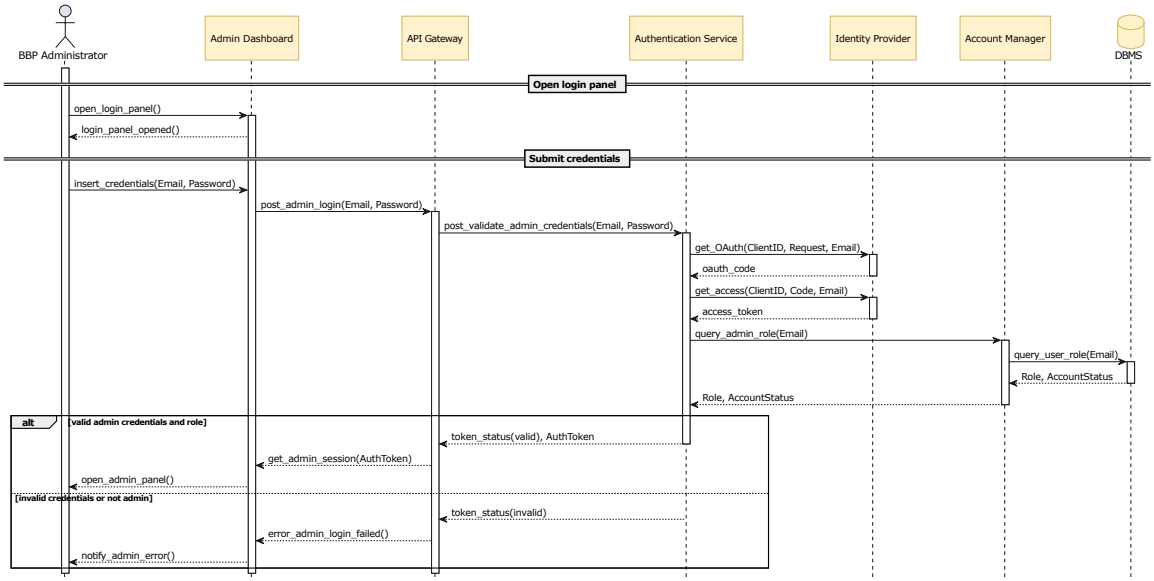


Figure 6: UC3 – Admin Login

The *Admin Login* use case describes the authentication process that allows a BBP administrator to access the administrative interface of the system. The interaction begins when the administrator opens the login panel from the **Admin Dashboard** and submits their credentials (email and password).
The credentials are forwarded to the **API Gateway**, which delegates their validation to the **Authentication Service**. The authentication process relies on the external **Identity Provider** to perform the OAuth-based authorization flow and obtain an access token. Once

the token is issued, the Authentication Service queries the **Account Manager** to verify that the authenticated user holds an administrative role and that the account is active. The Account Manager retrieves the role and account status information from the **DBMS**.
If the credentials are valid and the user is recognized as an administrator, the Authentication Service returns a valid authentication token to the API Gateway, allowing the Admin Dashboard to establish an authenticated admin session and grant access to the administrative panel. Otherwise, in case of invalid credentials or insufficient privileges, the login attempt is rejected and an error notification is displayed to the administrator.

2.4.4 UC4 – Configure Data Merging Policy

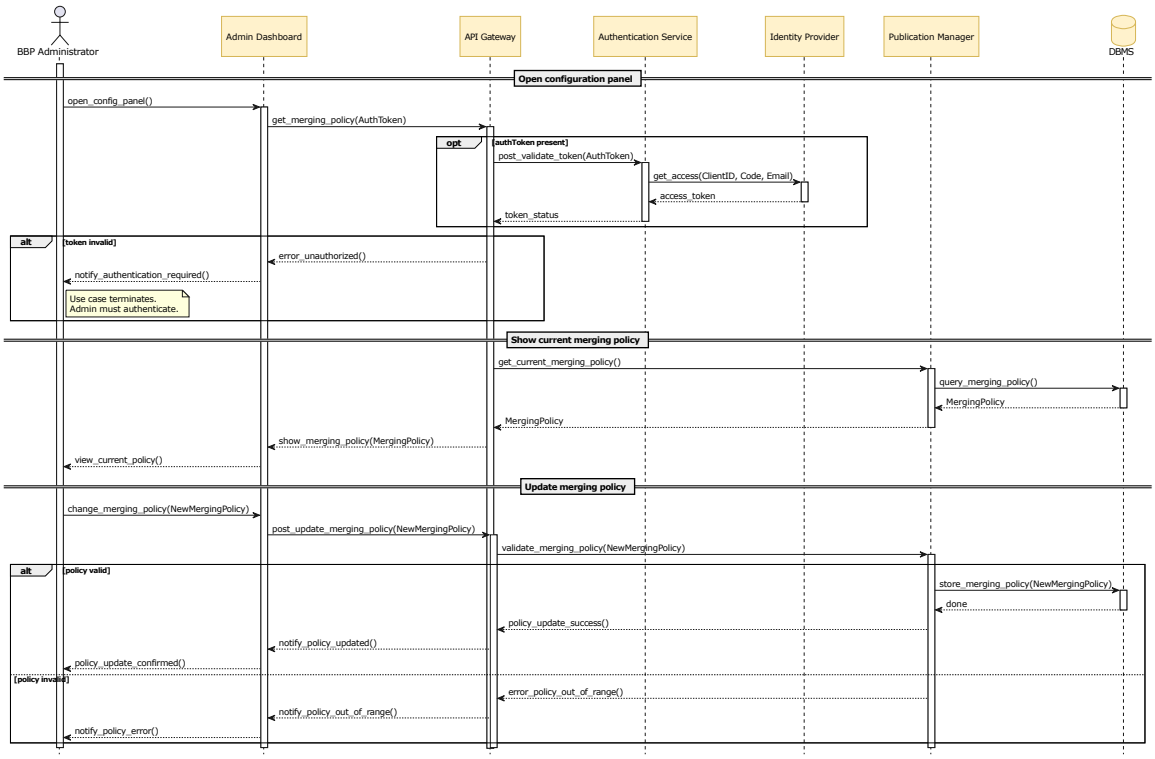


Figure 7: UC4 – Configure Data Merging Policy

The *Configure Data Merging Policy* use case describes how a BBP administrator manages the configuration rules used to merge path and trip data within the system. The interaction starts when the administrator accesses the configuration panel from the **Admin Dashboard**, which forwards the request to the **API Gateway**.
If an authentication token is provided, the API Gateway validates it through the **Authentication Service** and the external **Identity Provider**. In case of invalid credentials, the process is interrupted and the administrator is required to authenticate.
Once authorized, the **Publication Manager** retrieves the current merging policy from the **DBMS** and returns it to the Admin Dashboard for visualization. The administrator can then modify the configuration parameters and submit a new merging policy. The Publication Manager validates the updated policy and, if it satisfies the defined constraints,

persists it in the DBMS. A confirmation message is returned to the administrator upon successful update, while invalid configurations are rejected and reported as errors.

2.4.5 UC5 – Admin Edit Path

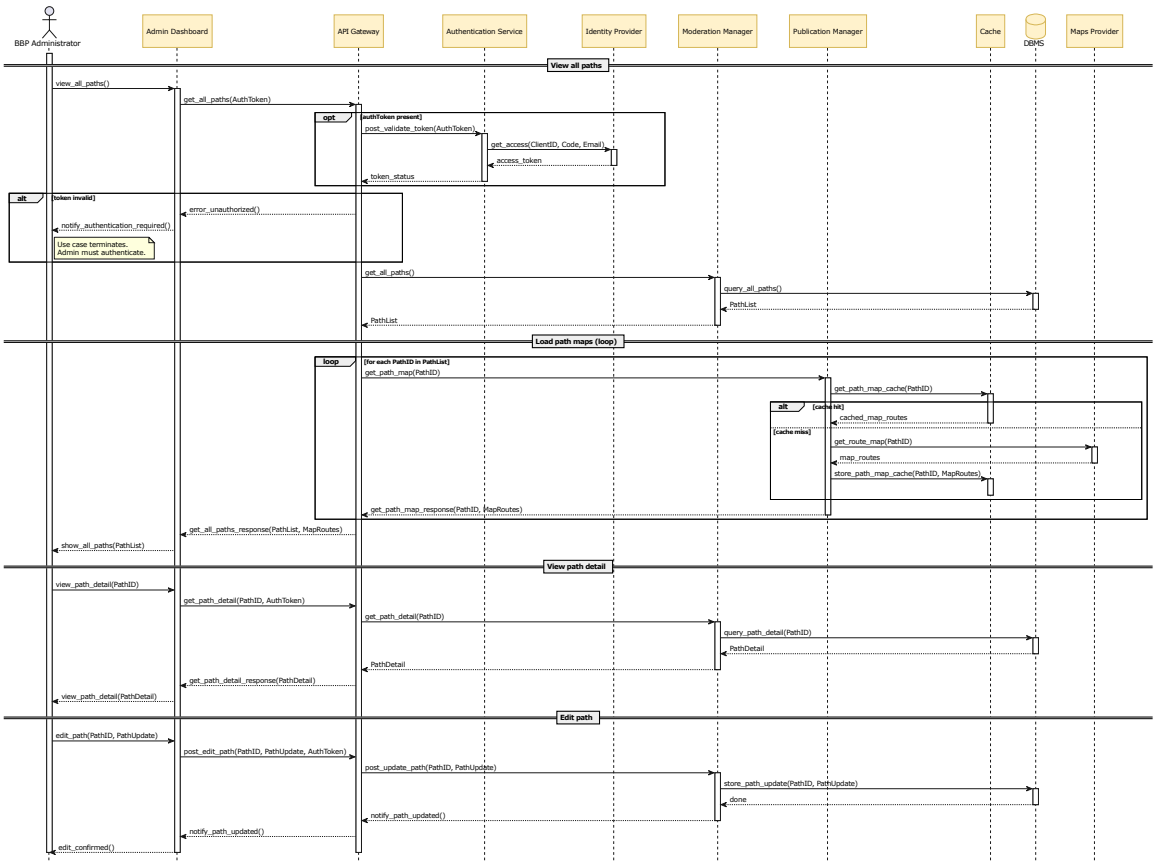


Figure 8: UC5 – Admin Edit Path

The *Manage Paths* use case describes the set of operations that allow a BBP administrator to visualize and edit all paths available in the system. The interaction starts when the administrator requests the list of all paths through the **Admin Dashboard**, which forwards the request to the **API Gateway**. If an authentication token is provided, the Gateway validates it through the **Authentication Service** and the external **Identity Provider**; invalid credentials cause the termination of the use case.

Once authorized, the **Moderation Manager** retrieves the list of all paths from the **DBMS**. For each path, the system loads the corresponding map representation using a cache-first strategy: cached map data is reused when available, while missing geometries are obtained from the external **Maps Provider** and stored in the **Cache**. The complete list of paths, enriched with their map representations, is then returned to the administrator.

The administrator may subsequently request detailed information for a specific path. The Moderation Manager retrieves the corresponding data from the DBMS and returns it to the Admin Dashboard for inspection. If the administrator decides to edit a path, the updated

information is validated and persisted by the Moderation Manager in the DBMS. Upon successful update, a confirmation message is sent back to the administrator, completing the management operation.

2.4.6 UC6 – Block User

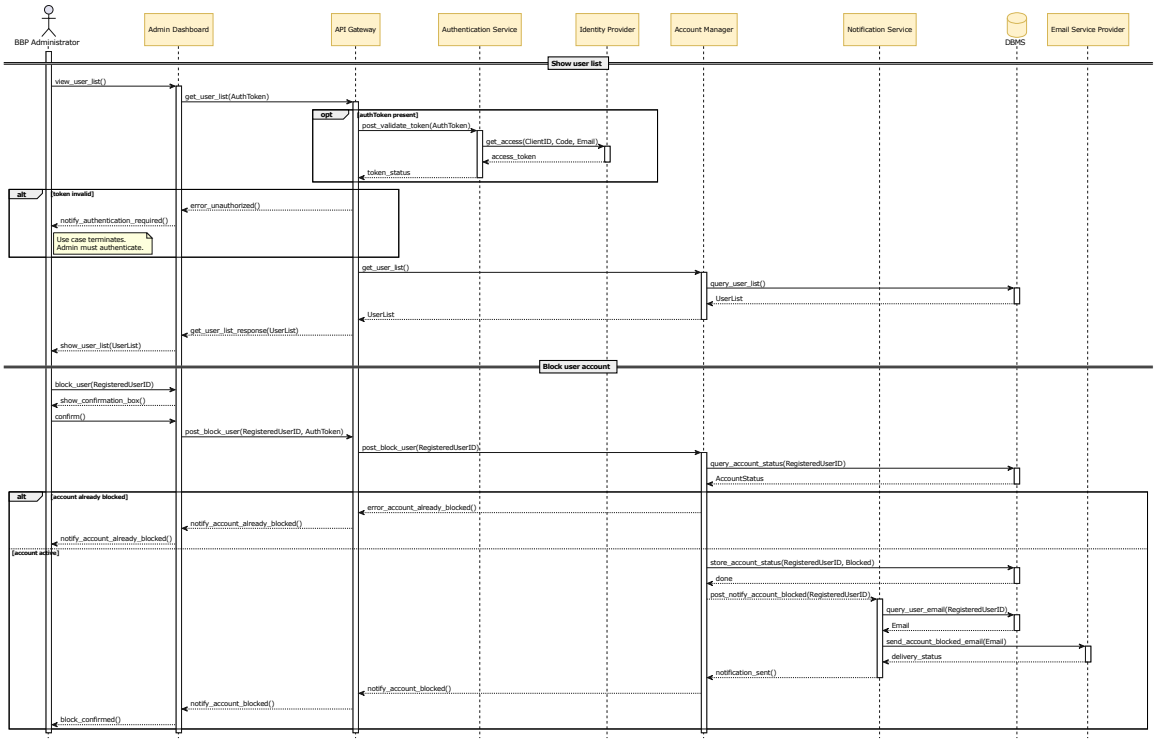


Figure 9: UC6 – Block User

The *Block User Account* use case describes the procedure through which a BBP administrator can block a registered user from accessing the system. The interaction begins when the administrator requests the list of users through the **Admin Dashboard**. The request is forwarded to the **API Gateway**, which validates the authentication token via the **Authentication Service** and the external **Identity Provider**. If authentication fails, the use case is terminated.

Once authorized, the **Account Manager** retrieves the list of registered users from the **DBMS** and returns it to the administrator. After selecting a specific user, the administrator confirms the intention to block the account. The Account Manager then verifies the current account status in the DBMS. If the account is already blocked, the administrator is notified accordingly.

If the account is active, the Account Manager updates the account status to *Blocked* in the DBMS. Subsequently, the **Notification Service** is invoked to inform the affected user. The Notification Service retrieves the user's email address from the DBMS and delegates the delivery of the notification to the external **Email Service Provider**. Upon successful completion, a confirmation is returned to the administrator, concluding the use case.

2.4.7 UC7 – User Registration

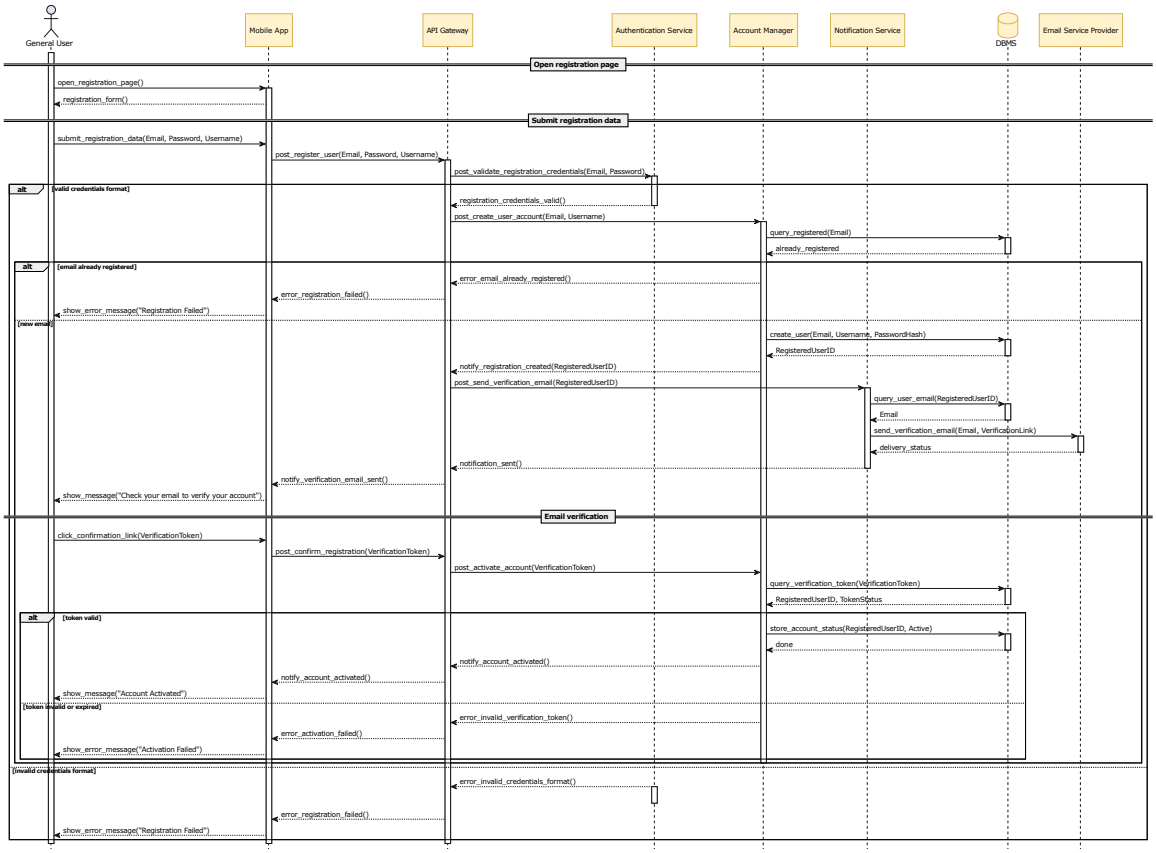


Figure 10: UC7 – User Registration

The *User Registration* use case describes the process through which a general user creates a new account within the system. The interaction starts when the user opens the registration page from the **Mobile App** and submits the required data, including email, username and password. The registration request is forwarded to the **API Gateway**, which delegates the validation of the credential format to the **Authentication Service**. If the credentials do not satisfy the required constraints, the registration is immediately rejected.

When the input is valid, the **Account Manager** is responsible for verifying whether the provided email address is already associated with an existing account by querying the **DBMS**. If the email is already registered, the registration attempt fails and an error message is returned to the user. Otherwise, a new user account is created and persisted in the DBMS with an inactive status.

After account creation, the **Notification Service** is invoked to send a verification email to the user. The service retrieves the user's email address from the DBMS and delegates the delivery of the verification message to the external **Email Service Provider**. The account activation is completed when the user follows the verification link contained in the email. The Account Manager validates the verification token and, if it is valid and not expired, updates the account status to active. In case of an invalid or expired token, the activation

process fails and the user is notified accordingly.

2.4.8 UC8 – User Login

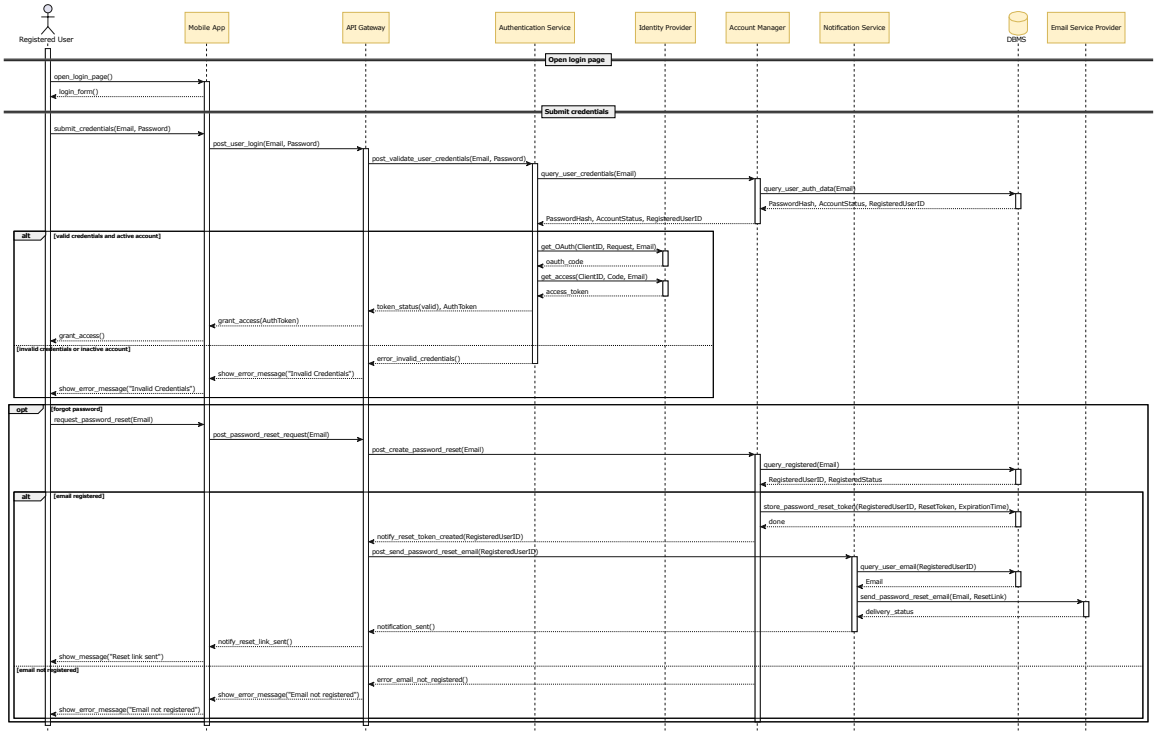


Figure 11: UC8 – User Login

The *User Login* use case describes how a registered user authenticates to the system and optionally initiates a password recovery procedure. The interaction starts when the user opens the login page from the **Mobile App** and submits their credentials. The request is forwarded to the **API Gateway**, which delegates credential validation to the **Authentication Service**. The Authentication Service queries the **Account Manager**, which retrieves authentication data from the **DBMS** and verifies both the password and the account status. If the credentials are valid and the account is active, the Authentication Service interacts with the external **Identity Provider** to obtain an OAuth authorization code and a corresponding access token. The token is then returned to the Mobile App, granting the user access to the system. If the credentials are invalid or the account is inactive, the login attempt fails and an error message is displayed.
The use case also includes an optional password recovery flow. When the user requests a password reset, the Account Manager generates a temporary reset token and stores it in the DBMS. The **Notification Service** retrieves the user's email address and sends a password reset link through the external **Email Service Provider**. The user is notified once the reset link has been successfully delivered.

2.4.9 UC9 – Record Trip

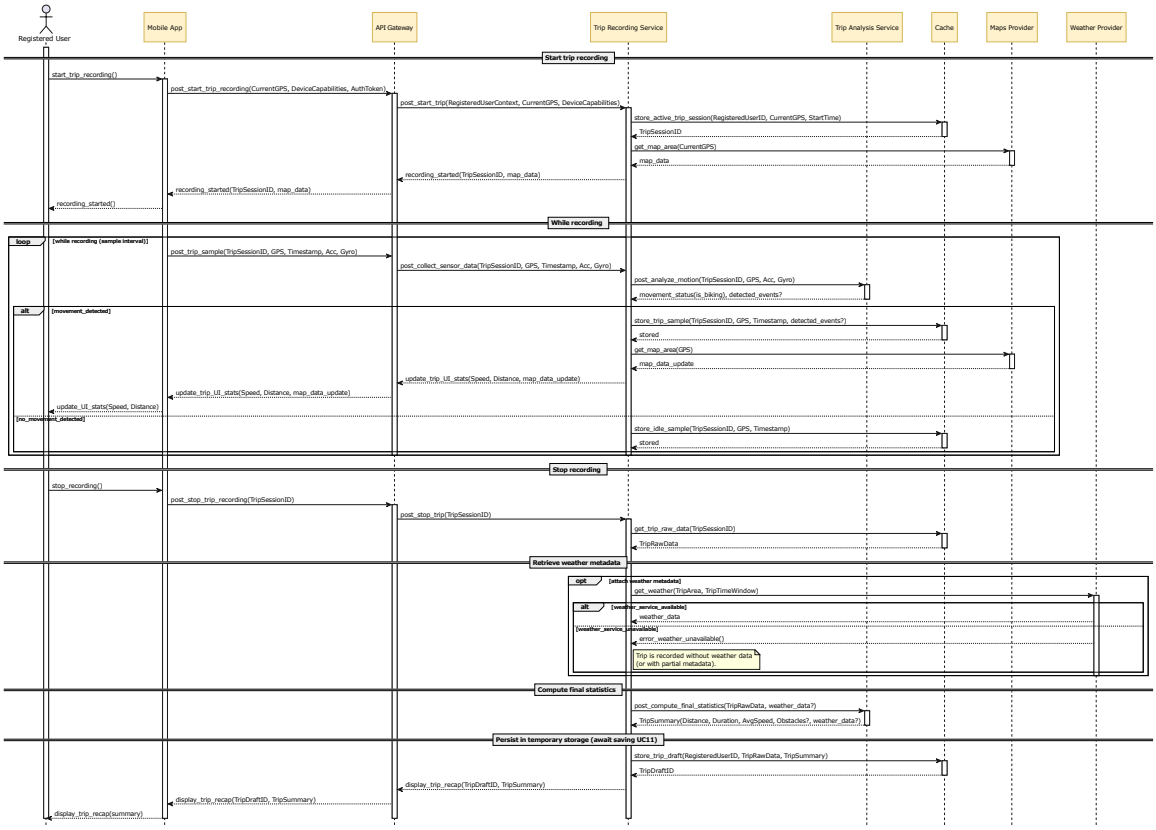


Figure 12: UC9 – Record Trip

The *Record Trip* use case describes how a registered user records a trip using the Mobile App, while the system collects sensor data and computes real-time and final statistics. The interaction begins when the user starts the recording. The Mobile App sends the current GPS position and device capabilities to the **API Gateway**, which forwards the request to the **Trip Recording Service**. A new active trip session is created and stored in the **Cache**, and the surrounding map area is retrieved from the external **Maps Provider** to initialize the user interface.

During the recording phase, the Mobile App periodically sends sensor samples containing GPS coordinates, timestamps, and inertial data (accelerometer and gyroscope). The Trip Recording Service forwards motion data to the **Trip Analysis Service**, which determines whether meaningful movement is occurring and detects relevant events. Movement-related samples are stored in the Cache, while idle samples are handled separately. When movement is detected, updated statistics and map data are sent back to the user interface in real time. When the user stops the recording, the Trip Recording Service retrieves the raw trip data from the Cache and optionally enriches it with weather metadata obtained from the external **Weather Provider**. The complete dataset is then analyzed to compute final trip statistics, such as distance, duration, average speed, and optional environmental information. The resulting trip is stored as a draft in temporary storage, awaiting a subsequent saving or

publication use case, and a recap of the recorded trip is displayed to the user.

2.4.10 UC10 – Manual Path Insertion

Figure 13: UC10 – Manual Path Insertion

The *Manual Path Insertion* use case describes how a registered user can manually create a new path by interacting with a map-based interface. The interaction begins when the user opens the manual insertion view from the **Mobile App**. The request is forwarded to the **API Gateway**, which delegates the operation to the **Manual Insertion Service**. The service retrieves the surrounding map area from the external **Maps Provider** and returns it to initialize the insertion interface.

During the route definition phase, the user incrementally selects points on the map. Each selected coordinate is temporarily stored in the **Cache** as part of a draft path. Optionally, for each newly added segment, the Manual Insertion Service requests geometric information from the Maps Provider to compute and visualize the corresponding path segment. The updated geometry is continuously returned to the Mobile App to provide real-time visual feedback.

Once the route geometry is defined, the user specifies additional path details such as sta-

tus and obstacles. These attributes are stored temporarily in the Cache and the user is prompted to confirm the creation of the path. Upon confirmation, the Manual Insertion Service retrieves the complete temporary path object, validates it, and persists it in the **DBMS**. If the validation succeeds, the user is notified of the successful path creation; otherwise, an error message is returned, concluding the use case.

2.4.11 UC11 – Store Trip

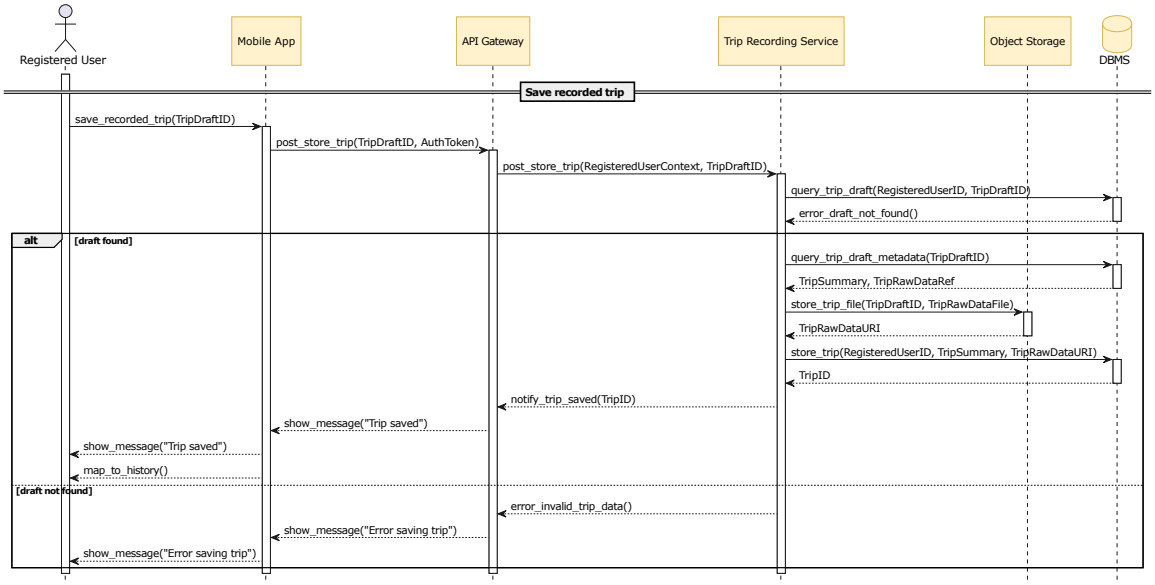


Figure 14: UC11 – Store Trip

The *Save Recorded Trip* use case describes how a registered user permanently stores a previously recorded trip. The interaction begins when the user selects a recorded trip draft from the **Mobile App** and requests to save it. The request is forwarded to the **API Gateway**, which delegates the operation to the **Trip Recording Service**.
The Trip Recording Service first verifies the existence of the specified trip draft by querying the **DBMS**. If the draft is not found, the saving process is aborted and an error message is returned to the user. If the draft exists, the service retrieves the associated metadata and raw trip data references. The raw trip data file is then stored in the **Object Storage**, and the resulting storage URI is obtained.
Finally, the Trip Recording Service persists the complete trip information in the DBMS, linking the stored metadata with the raw data location. Upon successful completion, the user is notified that the trip has been saved and the application redirects the user to the trip history view. In case of any error during the process, an appropriate failure message is displayed.

2.4.12 UC12 – Publish Path

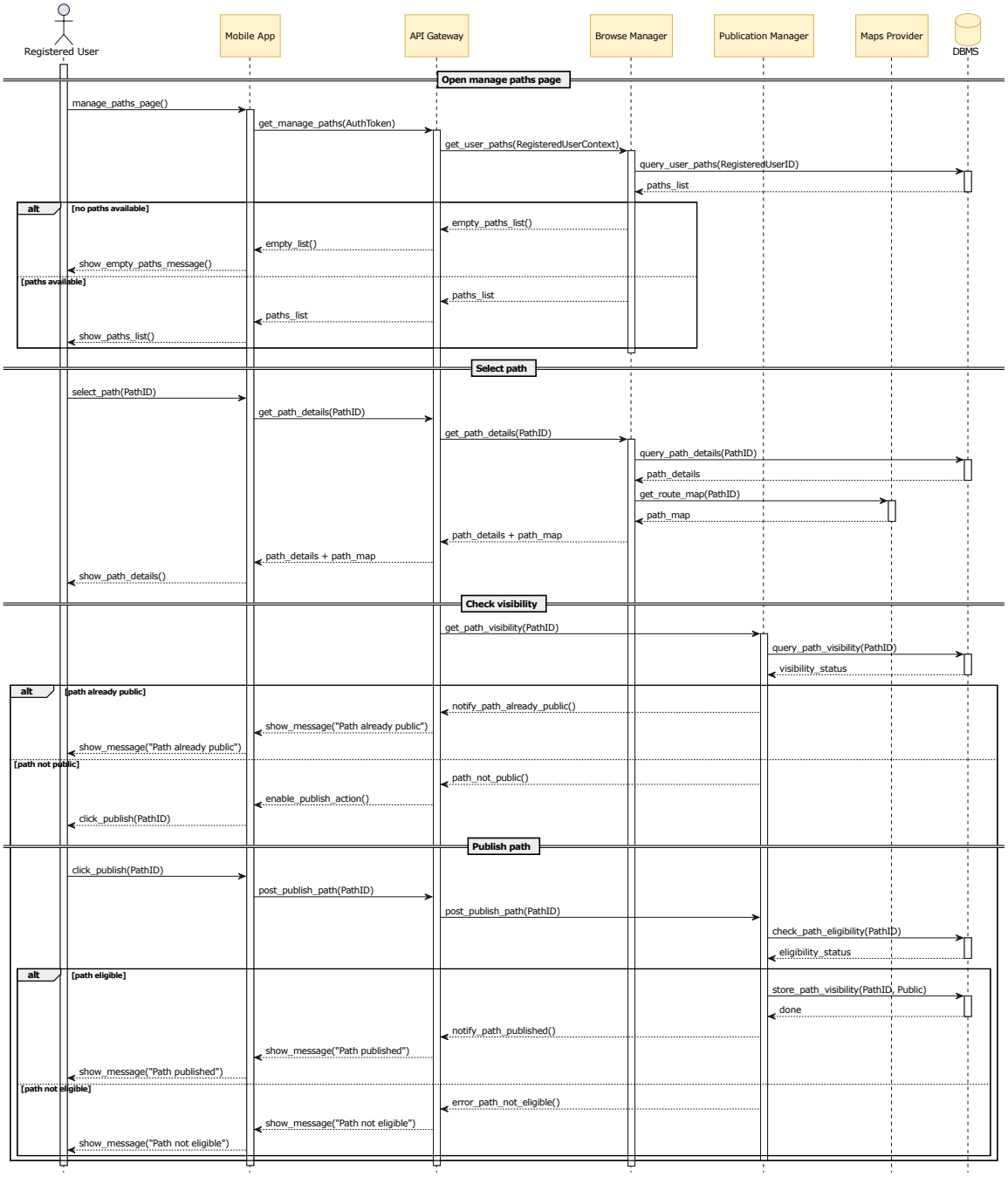


Figure 15: UC12 – Publish Path

The *Manage and Publish Paths* use case describes how a registered user manages the paths they have created and optionally publishes them. The interaction starts when the user opens the path management section from the **Mobile App**. The request is forwarded to the **API Gateway**, which delegates the retrieval of the user's paths to the **Browse Manager**. The Browse Manager queries the **DBMS** and returns the list of paths associated with the user. If no paths are available, an informative message is displayed.

After selecting a specific path, the user requests its details. The Browse Manager retrieves the path information from the DBMS and obtains the corresponding map representation from the external **Maps Provider**. The complete path details and map are then displayed to the user.
The system subsequently checks the publication status of the selected path through the **Publication Manager**. If the path is already public, the user is notified accordingly. Otherwise, the user is allowed to initiate the publication process. The Publication Manager verifies the eligibility of the path and, if the validation succeeds, updates its visibility status in the DBMS. A confirmation message is returned to the user upon successful publication, while non-eligible paths result in an error notification.

2.4.13 UC13 – View Trip History

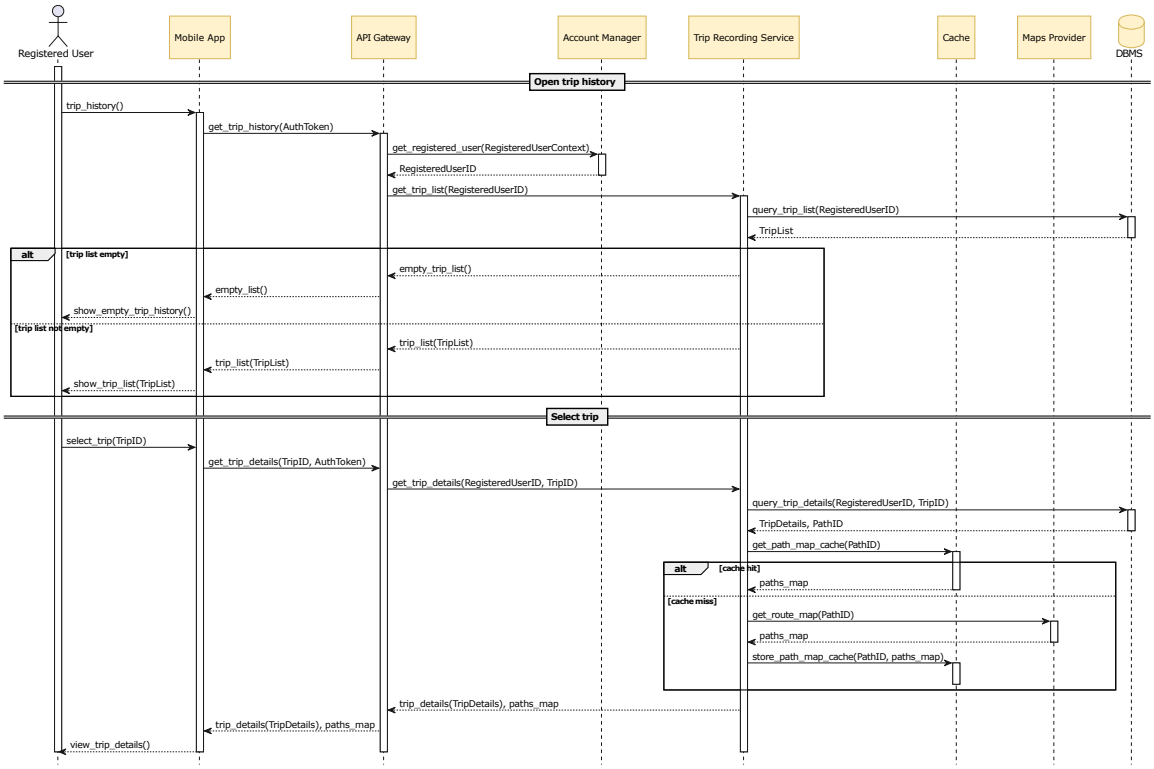


Figure 16: UC13 – View Trip History

The *View Trip History* use case describes how a registered user accesses and visualizes their previously recorded trips. The interaction starts when the user opens the trip history section from the **Mobile App**. The request is forwarded to the **API Gateway**, which retrieves the corresponding user identifier through the **Account Manager**.
Once the user is identified, the **Trip Recording Service** queries the **DBMS** to obtain the list of trips associated with the user. If no trips are available, the system notifies the user accordingly. Otherwise, the list of recorded trips is returned and displayed in the application.

When the user selects a specific trip, the Trip Recording Service retrieves detailed trip information from the DBMS. To visualize the associated route, the service attempts to obtain the path map using a cache-first strategy. If the map data is not available in the **Cache**, it is requested from the external **Maps Provider** and subsequently cached. The complete trip details, together with the map representation, are then returned to the user for inspection.

2.4.14 UC14 – Validate Automatically Created Path

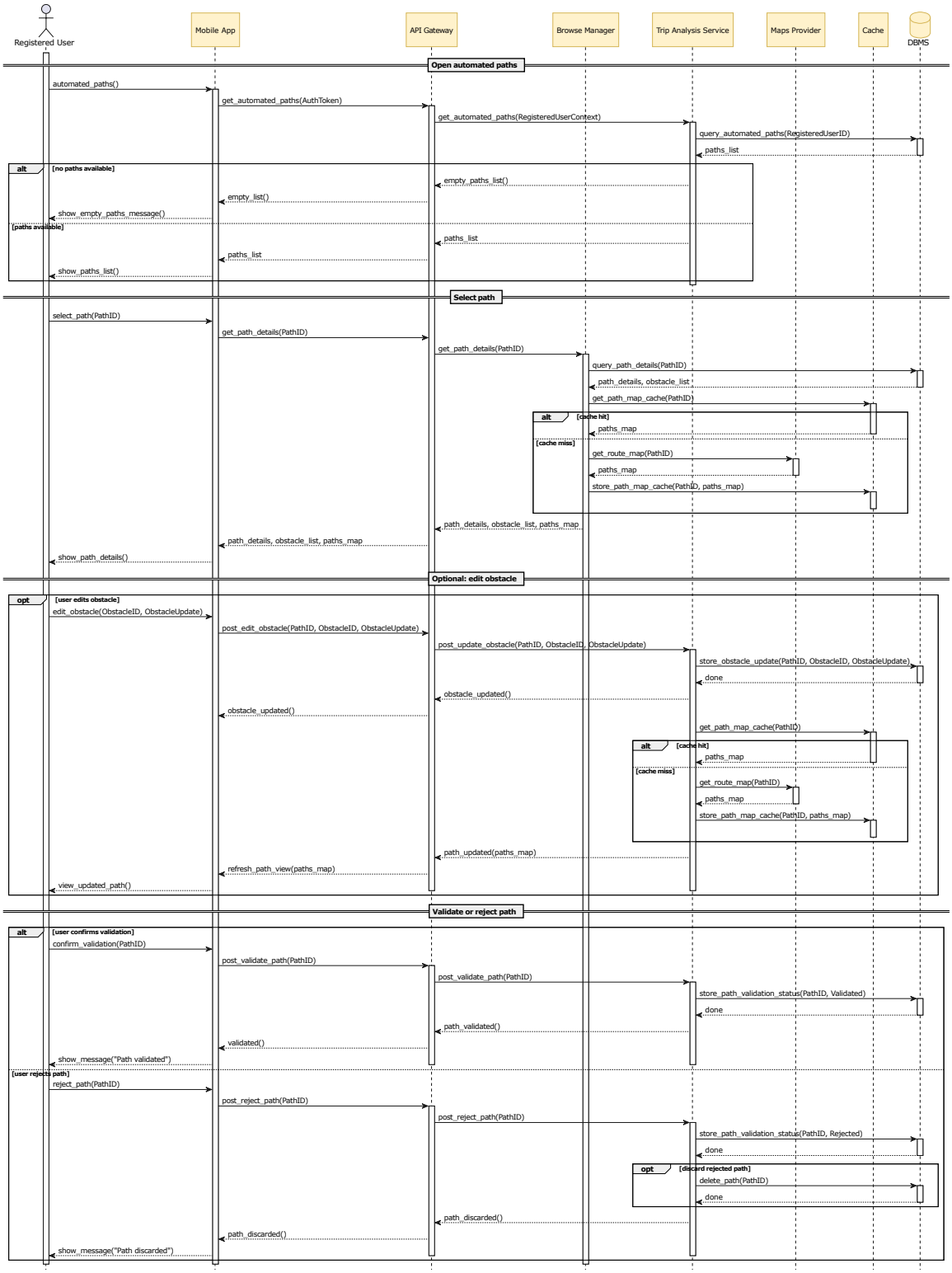


Figure 17: UC14 – Validate Automatically Created Path

The *Validate Automated Path* use case describes how a registered user reviews, edits, and validates paths that have been automatically generated by the system. The interaction

starts when the user accesses the list of automated paths from the **Mobile App**. The request is forwarded to the **API Gateway**, which delegates the retrieval of automated paths to the **Trip Analysis Service**. The service queries the **DBMS** and returns the list of available paths, or an empty result if none are present.
After selecting a specific path, detailed path information and associated obstacles are retrieved through the **Browse Manager**. The corresponding map representation is obtained using a cache-first strategy: cached map data is reused when available, while missing geometries are requested from the external **Maps Provider** and stored in the **Cache**. The complete path details are then displayed to the user.
Optionally, the user may edit detected obstacles. In this case, the updated obstacle information is stored by the Trip Analysis Service in the DBMS, and the path visualization is refreshed by retrieving updated map data from the cache or the Maps Provider. Once the review is complete, the user can either validate or reject the automated path. Validation results in the path being marked as validated in the DBMS, while rejection updates the path status accordingly and may optionally lead to its deletion. In both cases, the user is notified of the outcome, concluding the use case.

2.4.15 UC15 – Publish Trip

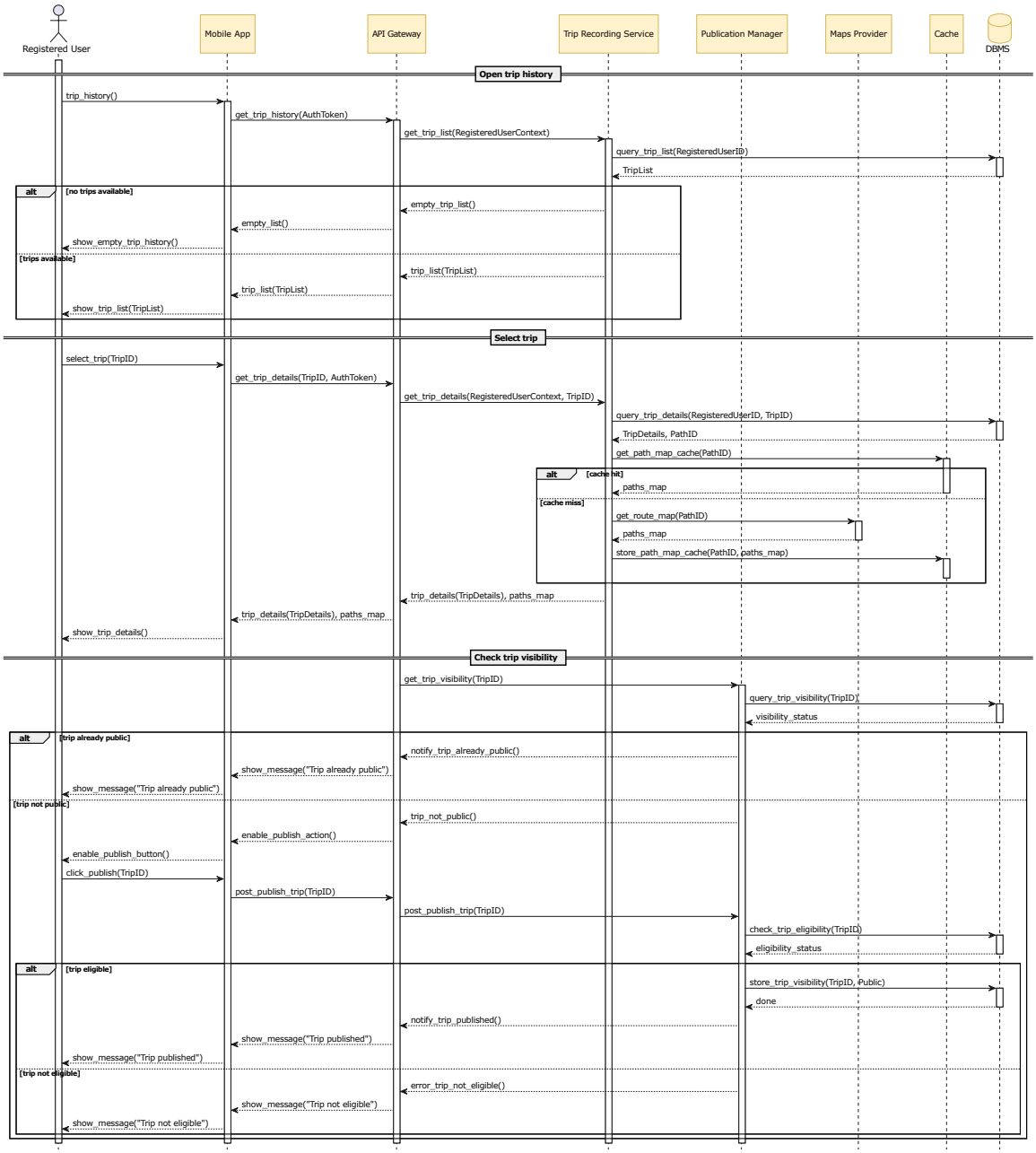


Figure 18: UC15 – Publish Trip

The *Publish Trip* use case describes how a registered user publishes one of their recorded trips, making it publicly visible within the system. The interaction starts when the user opens the trip history from the **Mobile App**. The request is forwarded to the **API Gateway**, which delegates the retrieval of the user's trips to the **Trip Recording Service**. The service queries the **DBMS** and returns either an empty result or the list of available trips.

After selecting a specific trip, detailed trip information is retrieved from the DBMS. To visualize the associated route, the Trip Recording Service applies a cache-first strategy:

map data is retrieved from the **Cache** when available, or requested from the external **Maps Provider** and subsequently cached. The trip details and map representation are then displayed to the user.
The system then checks the publication status of the selected trip through the **Publication Manager**. If the trip is already public, the user is notified accordingly. Otherwise, the user is allowed to initiate the publication process. The Publication Manager verifies the eligibility of the trip and, if valid, updates its visibility status in the DBMS. Upon successful publication, a confirmation message is returned to the user, while non-eligible trips result in an error notification.

2.4.16 UC16 – Manage Path (User Side)

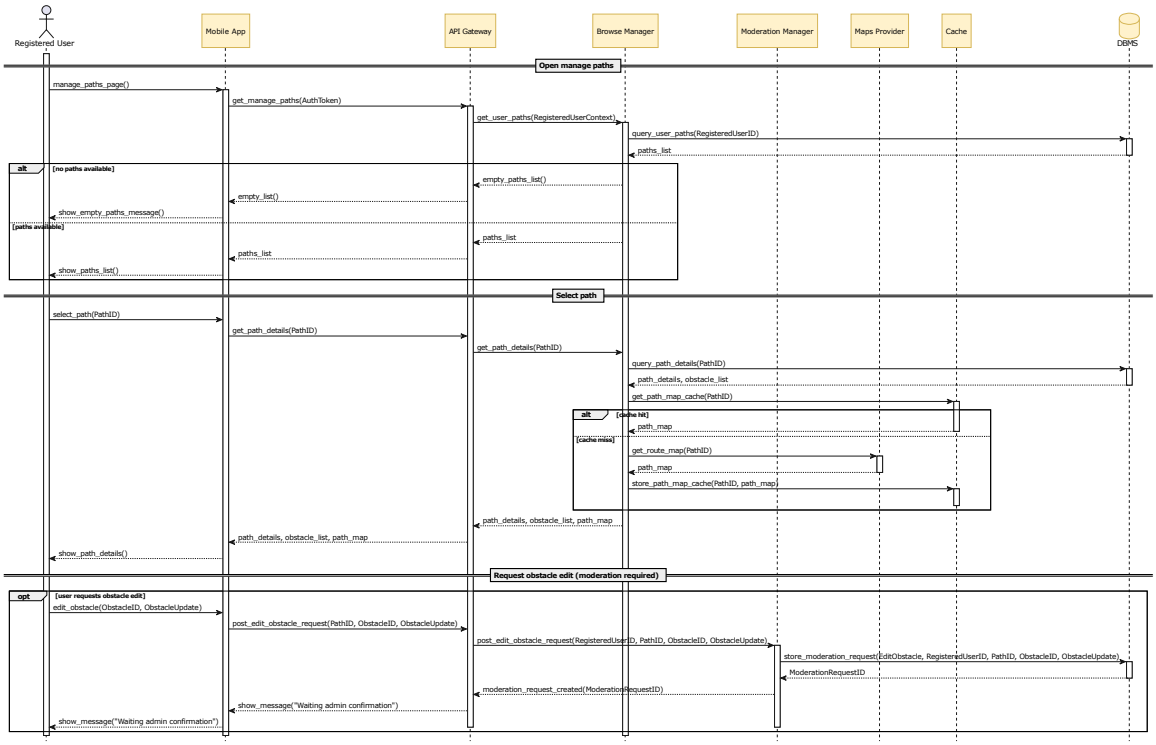


Figure 19: UC16 – Manage Path (User Side)

In the *Manage Path (User Side)* flow, a registered user accesses the management section from the **Mobile App** to view and modify paths they previously created. The request is handled by the **API Gateway**, which delegates the retrieval of the user's paths to the **Browse Manager**. The Browse Manager queries the **DBMS** to obtain the list of paths associated with the user and returns either an empty result or the available paths for visualization.
After the user selects a specific path, detailed information and related obstacles are retrieved from the DBMS. To support map visualization, the Browse Manager loads the path geometry using a cache-first strategy: it first queries the **Cache** and, if the map is not available, requests it from the external **Maps Provider** and stores it back into the cache.

The Mobile App then displays the path details, obstacles, and the corresponding map.
If the user requests an edit to an obstacle that requires moderation, the Mobile App submits an edit request to the API Gateway, which forwards it to the **Moderation Manager**. The Moderation Manager persists a moderation request in the DBMS, including the user identity, the target path and obstacle, and the proposed update. Once the request is stored, the user is notified that the modification is pending approval, concluding the user-side portion of the use case.

2.4.17 UC16 – Manage Path (Admin Side)

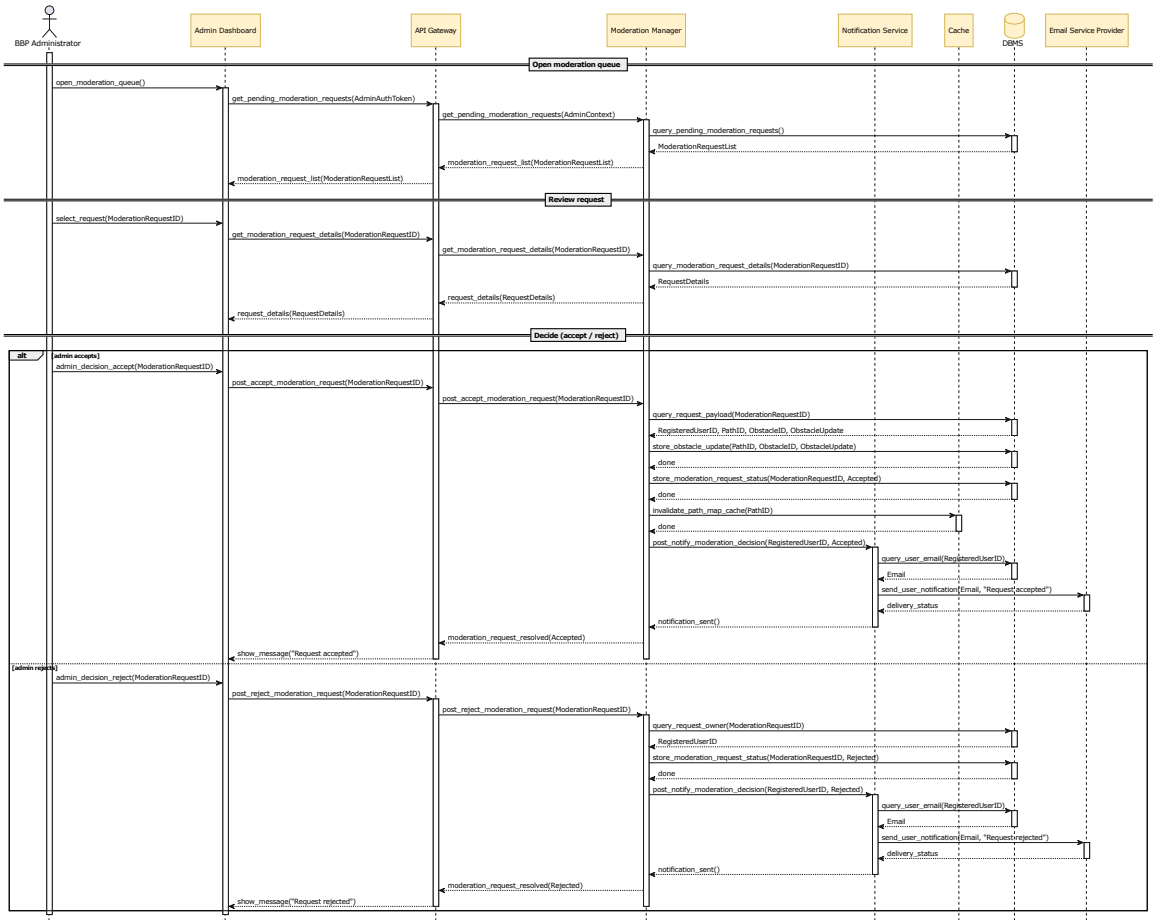


Figure 20: UC17 – Moderate Path (Admin Side)

In the *Manage Path (Admin Side)* flow, a BBP administrator processes user-submitted modification requests through the moderation interface. The interaction begins when the administrator opens the moderation queue from the **Admin Dashboard**. The request is forwarded to the **API Gateway**, which delegates the retrieval of pending items to the **Moderation Manager**. The Moderation Manager queries the **DBMS** and returns the list of pending moderation requests to be displayed in the dashboard.
When the administrator selects a specific request, the system retrieves the full request details from the DBMS, allowing the administrator to inspect the proposed change. The

administrator can then either accept or reject the request. In the acceptance case, the Moderation Manager loads the request payload (user, path, obstacle, and update), applies the obstacle update by persisting it in the DBMS, and marks the moderation request as accepted. To ensure consistency with map visualization, the path map entry is invalidated in the **Cache** so that subsequent visualizations will be regenerated using the updated data. In both acceptance and rejection cases, the moderation request status is stored in the DBMS. Finally, the **Notification Service** informs the user about the decision. It retrieves the user email address from the DBMS and delegates the message delivery to the external **Email Service Provider**. The outcome (accepted or rejected) is returned to the Admin Dashboard and shown to the administrator, concluding the moderation process.

2.5 Component Interfaces

This section summarizes the interfaces exposed by the system components and used for their interaction. Component interfaces are divided into two main categories:

- **API interfaces**, which define the operations exposed by each component to other services through REST calls
- **DBMS interfaces**, which define all interactions with the persistent storage layer and encapsulate database access operations.

All API interfaces follow a REST-oriented design and are used for inter-component communication within the application layer, while DBMS interfaces abstract data retrieval and manipulation, ensuring separation between business logic and data management. User interface logic, authorization checks, and error-handling details are not included, as this section focuses exclusively on the structural interfaces between components. To improve readability and avoid ambiguity in the interpretation of the listed interfaces, the following conventions and assumptions are adopted:

- API methods represent logical operations exposed by components and are invoked through REST calls mediated by the API Gateway.
- Method names are prefixed according to their semantic role: `get_*` for data retrieval, `post_*` for operations that create or trigger actions, and `put_*` for update operations.
- DBMS methods abstract one or more SQL operations executed on the persistent storage and are expressed in a parametric form, independently of the specific DBMS implementation.
- Methods prefixed with `query_*` correspond to read-only database operations, while `store_*`, `update_*`, and `delete_*` indicate write operations.
- Caching-related methods (e.g., `get__cache`, `store__cache`) are included in the DBMS interfaces, as they represent persistent or semi-persistent data access.
- User interface logic, presentation-level methods, and error-handling routines are intentionally excluded from this section.
- Authorization and authentication checks are assumed to be enforced by the API Gateway and are therefore not explicitly represented in the listed interfaces.

2.5.1 API Interfaces

API Gateway

- post_choose_path()
- post_insert_manual_trip()
- post_insert_path()
- post_login_admin()
- post_login_user()
- post_publish_path()
- post_publish_trip()
- post_save_recorded_trip()
- post_search_paths()
- post_signup_user()
- post_start_trip_recording()
- post_stop_trip_recording()
- post_validate_path()
- post_reject_path()
- post_create_moderation_request()
- post_accept_moderation_request()
- post_reject_moderation_request()

Authentication Service

- post_validate_admin_credentials()
- post_validate_user_credentials()
- post_validate_registration_credentials()
- post_validate_token()

Identity Provider

- get_access()
- get_user_info()

Account Service

- post_create_user()
- post_login()
- post_block_user()
- post_unblock_user()
- post_verify_email()
- post_send_password_reset()
- post_get_user_list()

Browse Service

- get_all_paths()

- get_user_paths()
- get_path_details()
- get_search_paths()
- post_choose_path()

Maps Service

- get_coordinates()
- get_route_map()
- get_trip_route_map()
- reverse_geocode()

Weather Service

- get_weather()

Trip Recording Service

- post_start_trip_recording()
- post_stop_trip_recording()
- post_save_recorded_trip()
- get_trip_history()
- get_trip_details()

Manual Insertion Service

- post_insert_manual_trip()

Publishing Service

- post_publish_path()
- post_publish_trip()
- post_update_merging_policy()
- post_retrieve_merging_policy()

Moderation Service

- post_create_moderation_request()
- post_accept_moderation_request()
- post_reject_moderation_request()
- get_pending_moderation_requests()
- get_moderation_request_details()

Email Service

- send_email()
- send_password_reset_email()
- send_verification_email()
- send_warning_email()

Notification Service

- send_user_notification()
- send_in_app_notification()

Analytics Service

- log_app_event()
- log_path_search()
- log_trip_started()
- log_trip_stopped()
- log_trip_saved()
- track_user_action()
- track_user_session()

2.5.2 DBMS Interfaces

- check_path_eligibility()
- check_trip_eligibility()
- query_account_status()
- query_user_list()
- query_user_email()
- query_all_paths()
- query_user_paths()
- query_candidate_paths()
- query_automated_paths()
- query_path_details()
- query_path_visibility()
- query_path_info()
- query_trip_list()
- query_trip_details()
- query_trip_draft()
- query_trip_draft_metadata()
- query_moderation_request_details()
- query_pending_moderation_requests()
- query_request_payload()
- query_request_owner()
- query_request_user_id()
- query_merging_policy()
- get_path_map_cache()
- get_path_info_cache()
- get_trip_map_cache()
- get_search_cache()
- get_weather_cache()

- store_user_credentials()
- store_user_profile()
- store_user_block_status()
- store_user_email_verification_status()
- store_user_password_reset_token()
- store_path()
- store_path_visibility()
- store_path_validation_status()
- store_path_map_cache()
- invalidate_path_map_cache()
- store_trip()
- store_trip_visibility()
- store_trip_map_cache()
- store_trip_file()
- store_trip_raw_data_uri()
- store_trip_summary()
- store_manual_trip()
- store_moderation_request()
- store_moderation_request_status()
- store_obstacle_update()
- store_search_cache()
- store_weather_cache()
- delete_path()
- update_merging_policy()
- update_user_password()
- verify_email_token()
- verify_admin_confirmation_token()

2.6 Selected Architectural Styles and Patterns

The BBP system is designed according to a 4-tier architectural style, which allows a clear separation of concerns, scalability, and ease of maintenance. Each tier is responsible for a well-defined set of functionalities and communicates with adjacent layers through well-defined interfaces.

The four tiers are:

- Presentation Tier
- Application Tier - Web Layer
- Application Tier - Backend Layer
- Data Tier

This architectural choice is consistent with the requirements identified in the RASD, especially those related to scalability, availability, integration with external services, and data aggregation.

2.6.1 Presentation Tier

The Presentation Tier is responsible for managing user interaction with the BBP system.
It includes:

- Mobile applications
- Web-based interfaces accessible through a browser

This tier allows both registered and unregistered users to:

- Register and authenticate
- Record trips
- Browse routes and path information
- Visualize maps, paths, and statistics

The Presentation Tier does not contain business logic.
It only handles:

- User input validation
- Data visualization
- Communication with the Application Tier via REST APIs

2.6.2 Application Tier - Web Layer

The Web Layer acts as a coordination layer between the Presentation Tier and Backend Layer.
Its responsibilities include:

- Managing user sessions and authentication flows
- Load balancing, distributing incoming requests among multiple Business Logic service instances
- Basic traffic control mechanisms, including request throttling and preliminary filtering to mitigate abusive or excessive requests

This tier exposes RESTful APIs consumed by the Presentation Tier and delegates all domain-specific decisions to the Backend Layer.

2.6.3 Application Tier - Backend Layer

The Backend Layer implements the Business Logic Tier of the BBP system and is designed following a microservices-based architecture. Each microservice encapsulates a well-defined business responsibility and can be deployed, scaled, and maintained independently, improving modularity, scalability, and fault isolation. All backend services are accessed exclusively through the Web Layer and do not interact directly with client applications. Communication between services occurs through well-defined APIs, while persistent data access is delegated to the Data Tier.

The Backend Layer is responsible for:

- Enforcing all business rules of the BBP system
- Processing trip data and computing statistics
- Managing path validation, publication, and aggregation
- Ensuring data consistency and quality
- Interacting with external services
- Supporting scalability and fault isolation through service separation

2.6.4 Data Tier

The Data Tier is responsible for persistent data storage.

It includes:

- Databases storing users, trips, paths, obstacles, and aggregated path data
- Access mechanisms abstracted through repositories or data access services

The Data Tier ensures:

- Data integrity
- Secure storage of sensitive information
- Efficient querying for route computation and aggregation

No other tier directly accesses the database.

2.7 Other Design Decisions

The BBP system interacts with multiple external services, which are essential for providing complete and accurate functionalities.

2.7.1 External Map Service

The BBP system relies on external map services to provide geographic information and map visualization features. These services are used to display maps, reconstruct recorded paths, and compute routes between an origin and a destination.
The interaction with external map services is handled by the Backend Layer, which requests map data such as street geometry, coordinates, and routing information. This approach keeps the Presentation Tier independent from map service implementation details and allows the system to change or extend the map provider without affecting the user interface.
Map data is used for:

- Visualizing paths and routes on the map
- Supporting route search and ranking
- Associating geographic positions with trips and path segments

To improve performance, frequently accessed map-related data may be cached by the system, reducing the number of external requests and ensuring faster response times.

2.7.2 External Weather Service

To enrich trips and paths with environmental context, the BBP system integrates with an external Weather Service. This service provides meteorological data such as temperature, wind conditions, and general weather status for a given geographic area and time window. Weather data is accessed by backend services, mainly during trip recording and analysis phases. The integration is designed to be optional and fault-tolerant: if the Weather Service is unavailable, trips can still be recorded and stored, possibly with partial or missing weather metadata. This ensures robustness and continuity of the core functionalities while enhancing the user experience when external data is available.

2.7.3 Device Sensor Interfaces

The BBP system relies on device-level sensors available on user smartphones, including:

- GPS
- Accelerometer
- Gyroscope

These sensors provide raw data used for:

- Automatic trip recording
- Path reconstruction
- Obstacle detection

Sensor data is collected at the Presentation Tier and processed by the Backen Layer.

2.7.4 Communication Interfaces and Interaction Model

Communication within the BBP system primarily follows a Client–Server model based on RESTful APIs over HTTP/HTTPS. The Presentation Tier interacts with the Application Tier through stateless REST calls, exchanging data in a standardized format JSON. This choice promotes interoperability, scalability, and ease of integration across different client platforms.

Within the Backend Layer, which is structured as a set of microservices, interactions are not limited to synchronous request–response patterns. Several components communicate using an event-driven approach.

This interaction model reduces coupling between microservices, improves scalability, and allows the system to react to internal state changes efficiently. Overall, the combination of REST-based client communication and event-driven backend interactions provides a flexible and robust communication infrastructure aligned with the architectural views presented in this document.

3 User Interface Design

The user interface lifecycle is presented in the UX diagram below. The design focuses on usability, readability, and intuitive navigation, strictly adhering to the requirements defined in RASD 3.1.1.

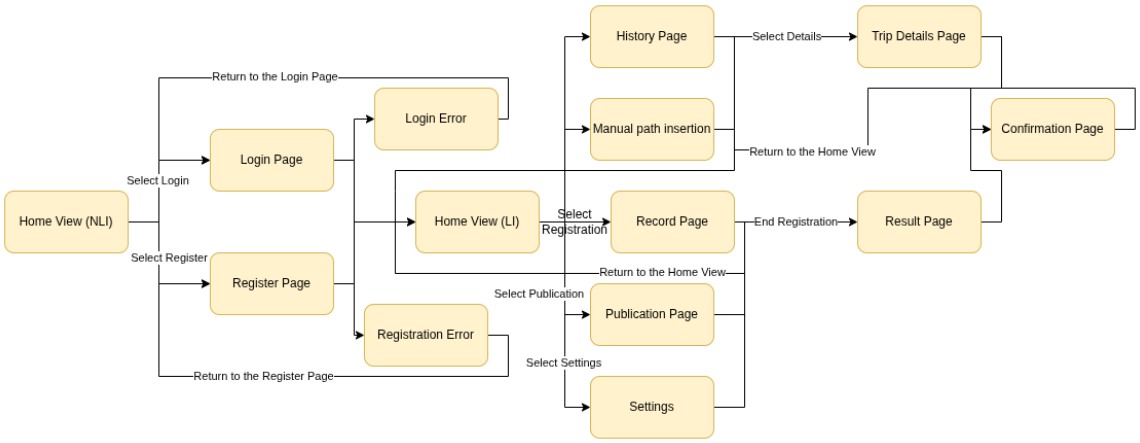


Figure 21: User Interface Diagram

Initially, all users enter the system as General Users, and the application displays the `Home View (NLI)`, where *NLI* stands for *Not Logged In*. At this stage, the user can browse paths, but the navigation bar is disabled, and advanced features are unavailable. The General User can register or log in to unlock full functionality. If the authentication process fails, the user can retry; otherwise, upon success, the user transitions to the Registered User role. Consequently, the application switches to the `Home View (LI)` (*Logged In*). The navigation bar becomes active, allowing access to the `History Page`, `Manual Path Insertion`, `Record Page`, `Publication Page`, and `Settings`.

3.1 Home View

Figure 22: Home View (LI) and (ULI)

The view above presents the concept of the app's Home Page. The application is shown in **light mode** with default settings. The screen contains a central map, a top bar with the app's name and login buttons, a path browsing panel, and a navigation bar at the bottom. Users can browse paths by simply entering an origin and destination. The application displays suitable routes or provides recommendations if no path is found.
For users who are not logged in, the navigation bar is grayed out and disabled, and a warning is shown at the top. The navigation bar provides access to views available only to registered users.

3.2 Record Page

Figure 23: UI: Record Page

The `Record Page` is a dedicated interface for capturing trips in Automatic Mode. It is designed to provide legibility during physical activity and consists of three main areas.
Map Layer: The central component visualizes the user's current position.
Telemetry Dashboard: Fixed at the top of the screen, this panel provides immediate feedback on the trip's progress. It displays live data: Current Status (Recording/Paused), Current Speed, Average Speed, and Elapsed Duration.
Control Panel:

- Pause/Resume - A yellow button to temporarily suspend data trip.
- Stop - A red button to permanently finish the recording.

When the user presses the Stop button, the system first requests confirmation to prevent

accidental cancellation. Upon confirmation, the application transitions to the `Result Page`, presenting the summary of the recorded path. Then it proceeds to the `Confirmation Page` allowing user to confirm path.

3.3 History Page

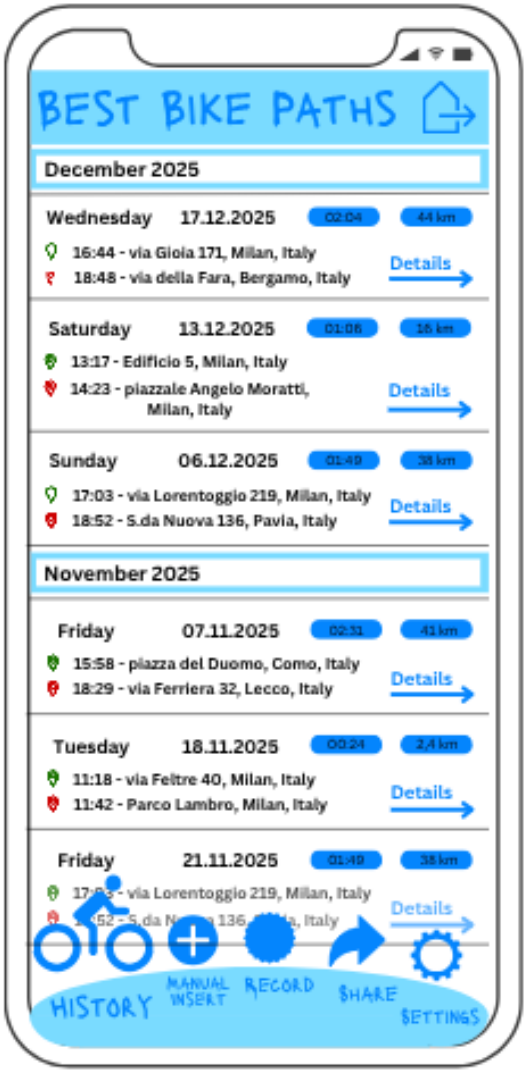


Figure 24: UI: History Page

The `History Page` serves as a persistent archive of the user's activity. To facilitate navigation, trips are aggregated chronologically and grouped by month.
Each list item acts as a summary card displaying key metrics:

- Path - origin and destination points and length.
- Timing - precise start/end timestamps and total duration.

The list supports a drill-down interaction model. Tapping on a specific trip summary redirects the user to the `Trip Details Page`, revealing full telemetry data and map visualization. If the detailed trip is unconfirmed, user can confirm it for this stage.

3.4 Manual path insertion

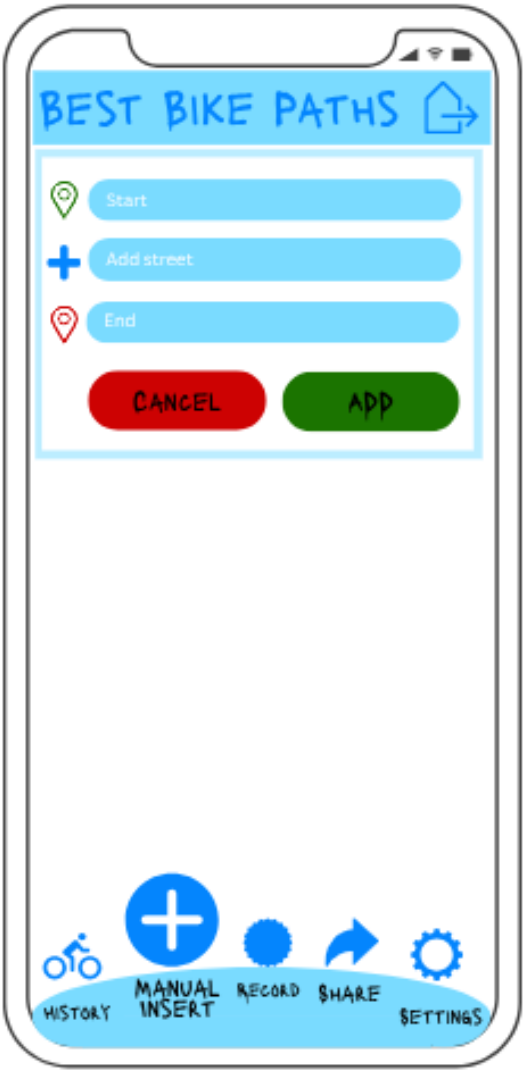


Figure 25: UI: Manual Insertion

The `Manual Path Insertion` interface allows for manual route creation. The users provide the origin, destination, and all intermediate streets. The `Add` button expands the view to input streets sequentially. Clicking `Cancel` restarts the process, whereas the final `Confirm` button adds the path to the user's profile.

3.5 Publication Page

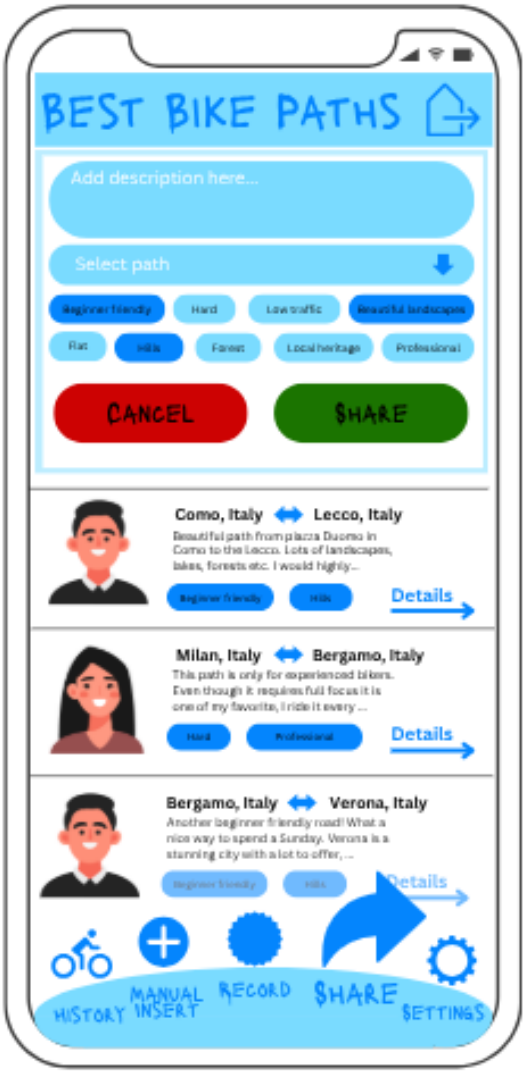


Figure 26: UI: Publication Page

The `Publication` page, located within the `Share` view, manages social interactions between users of the BBP app. The interface is divided into two sections. The upper panel is dedicated to publishing paths. To share a route, the user must select a confirmed path from their history, add a short description, and assign relevant tags. The lower panel displays a feed where the user can browse and scroll through posts from other community members.

3.6 Settings Page

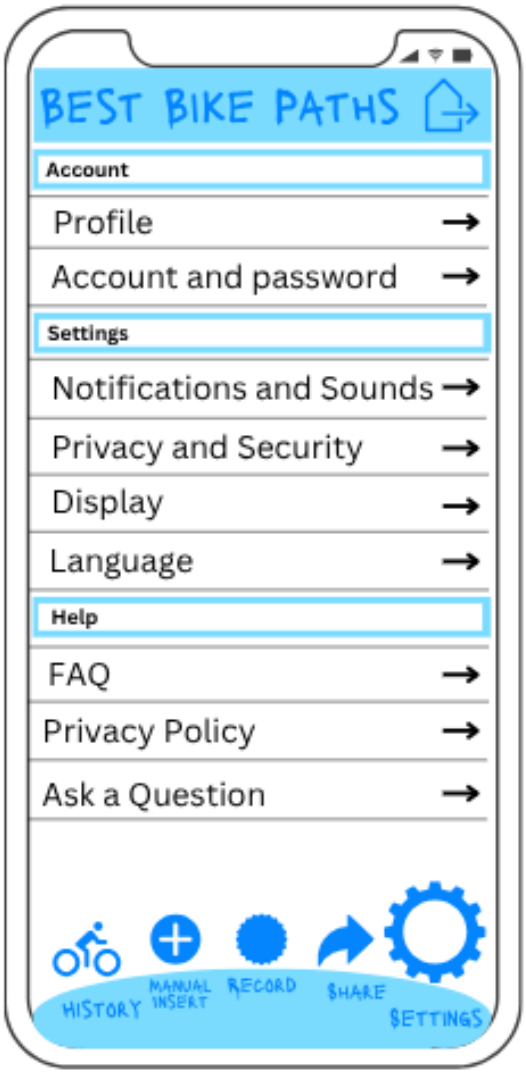


Figure 27: UI: Settings Page

The `Settings` page provides a centralized interface for application configuration. Users can update their profile details and account credentials, as well as customize the app's appearance and language preferences. Additionally, this section allows one to manage notification settings and access the help center for support.

4 Requirements Traceability

4.1 User Registration and Account Management

Requirements	Components
• **R1:** Guest user registration • **R2:** Email confirmation • **R3:** User login • **R4:** Password reset • **R5:** Deny invalid access • **R6:** View and update profile • **R7:** Delete account	• Mobile App • API Gateway • Authentication Service • Account Manager • Notification Service • DBMS • Identity Provider

4.2 Trip Recording, Statistics and Publication

Requirements	Components
• **R8:** Start, pause and stop trip recording • **R9:** Collect GPS and sensor data • **R10:** Detect biking activity • **R11:** Retrieve weather data • **R12:** Store completed trip • **R13:** Browse trip history • **R14:** Publish trip statistics	• Mobile App • API Gateway • Trip Recording Service • Trip Analysis Service • Publication Manager • Cache • DBMS • Weather Service • Maps Service

4.3 Path Contribution and Validation

Requirements	Components
• **R15:** Manual path creation • **R16:** Edit or delete manual paths • **R17:** Detect obstacles from sensors • **R18:** Show validation summary • **R19:** Confirm or reject obstacles	• Mobile App • API Gateway • Manual Insertion Service • Trip Analysis Service • Moderation Manager • DBMS • Maps Service

4.4 Path Search and Routing

Requirements	Components
• **R20:** Search paths by origin and destination • **R21:** Compute routes • **R22:** Rank routes • **R23:** Filter search results • **R24:** Visualize routes on map	• Mobile App • API Gateway • Browse Manager • Cache • DBMS • Maps Service

4.5 Data Aggregation and Governance

Requirements	Components
• **R25:** Aggregate user contributions • **R26:** Resolve conflicts • **R27:** Administrator login • **R28:** Configure merging policies • **R29:** Moderate content	• Admin Dashboard • API Gateway • Moderation Manager • Authentication Service • DBMS

5 Implementation, Integration & Test Plan

5.1 Overview and Implementation Plan

This chapter describes how the Best Bike Paths (BBP) system will be implemented, integrated, and tested. The goal of this plan is to define a clear strategy for building the system incrementally, while continuously verifying the correctness of its components and their interactions.

The implementation is performed in an incremental way. Components are not developed and integrated all at once, but progressively, following a predefined build plan. Testing activities are carried out in parallel with implementation in order to detect errors as early as possible and to simplify fault localization.

A combination of **bottom-up** and **thread-based** strategies is adopted. The bottom-up strategy defines the order in which components are implemented and integrated. Development starts from low-level components that do not depend on other modules and can be tested in isolation. As new components become available, they are integrated step by step, forming progressively larger and more complete subsystems.

In particular, backend services are implemented and tested individually before being combined into higher-level orchestration flows.

The thread-based strategy is used to guide the implementation from a functional perspective. A thread represents a user-visible functionality that involves multiple components across different layers of the system. By following this approach, it is possible to incrementally build and evaluate complete execution flows, while still respecting the dependency constraints imposed by the bottom-up strategy.

Testing activities follow the principles of the V-model. Unit and component tests are executed during the development of individual modules to verify their behavior in isolation. Integration testing is performed whenever new components are combined, focusing on interface correctness, data flow, and interaction among modules. During early integration stages, test drivers are used to simulate missing components and are progressively replaced as real modules are integrated.

External services are not unit tested. Their correct usage is verified through integration and contract-based tests, ensuring proper interaction with external APIs and correct handling of failure scenarios.

Once complete execution threads are integrated, system-level tests are performed to verify the correct behavior of the system from the user perspective. This approach ensures a controlled and progressive construction of the system, while maintaining continuous verification throughout the development process.

5.2 Features Identification

The implementation process is organized around a set of core system features derived from the functional requirements and use cases defined in the RASD. Features are implemented incrementally, taking into account dependencies among backend services and prioritizing components that are required by multiple workflows.

[F1] Login and Registration
Login and Registration are implemented first, as they are required by most system functionalities. This feature includes user login and registration workflows, integration with an external Identity Provider using OAuth and OpenID Connect, validation of access tokens, and basic role-based access control.

[F2] Path Browsing and Visualization
Path browsing functionalities are implemented once basic user management is in place. This feature allows users to retrieve and visualize available paths. The Browse Manager is responsible for accessing path data from the DBMS, interacting with external Maps services to handle geographic information, and applying caching mechanisms to reduce response time. This functionality supports both authenticated and unauthenticated users.

[F3] Trip Recording
Trip recording is implemented as a feature mainly targeting mobile clients. The Trip Recording Service manages the lifecycle of a trip, including its creation, data collection from sensors, and temporary storage of raw samples. During recording, intermediate data may be processed to compute basic statistics and to support responsive feedback to the user.

[F4] Manual Path Insertion
Manual path insertion allows users to explicitly define new paths through map-based interaction. This feature is implemented after browsing and mapping services are available. The Manual Insertion Service validates user-provided geometry and metadata and stores validated paths in the DBMS. External Maps services are used to support route definition and visualization.

[F5] Search and Ranked Paths
This feature allows users to search for paths and obtain ranked results based on quality indicators computed from historical trip data. The Browse Manager coordinates the search process, applying filtering criteria and requesting ranking information derived from the Trip Analysis Service. Ranked paths are retrieved from the DBMS and returned to the client in a sorted order. This functionality is available to both authenticated and unauthenticated users and represents an advanced read-oriented feature built on top of previously collected data.

[F6] Admin Path Management
This feature allows administrators to view and edit existing paths. The system retrieves path data, supports map-based visualization, and applies validation rules when path updates are submitted. Administrative edits are stored persistently and may affect the visibility or eligibility of paths for publication. This feature builds upon previously integrated browsing, caching, and authentication functionalities.

[F7] Path Publication
This feature allows users to publish newly created paths and to update existing ones. The system validates path data, checks publication constraints, and updates the visibility state of the path accordingly. This feature represents the final step in the path lifecycle and depends on previously integrated browsing and insertion functionalities.

5.3 Component Integration and Testing

This section describes the component integration and integration testing strategy adopted for the system. Components are integrated incrementally as new features are introduced, and each integration step is validated through dedicated integration tests.
The integration process follows a bottom-up approach. Low-level components are integrated and tested first, while higher-level components are added only after their dependencies have been validated.
During early stages, test drivers simulate client requests (Mobile App / Admin Dashboard) and trigger the target services. The DBMS and the database are assumed to be available during all integration steps and are therefore not explicitly represented in the integration diagrams. External services are also omitted, as they are not part of the system and are assumed to operate independently. The following subsections describe the integration steps associated with each system feature.

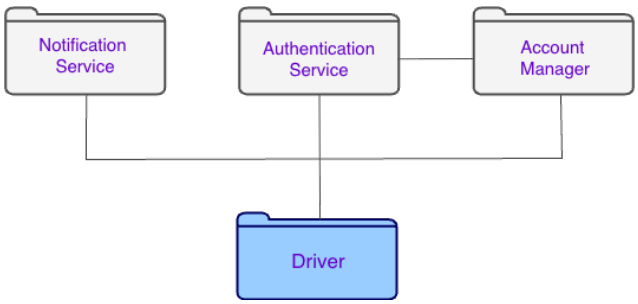


Figure 28: [F1]: Login and Registration

This first step validates the authentication workflows (login and registration) and establishes a trusted security baseline for the following tests. The Driver invokes the *Authentication Ser-*

vice and the *Account Manager*; for registration, the *Notification Service* is also involved to support verification emails. The *Identity Provider* is logically required for OAuth/OpenID Connect flows, but it is treated as an external dependency and therefore not represented in the integration diagram. After this step, the *Authentication Service* is considered stable and reused in all subsequent steps.

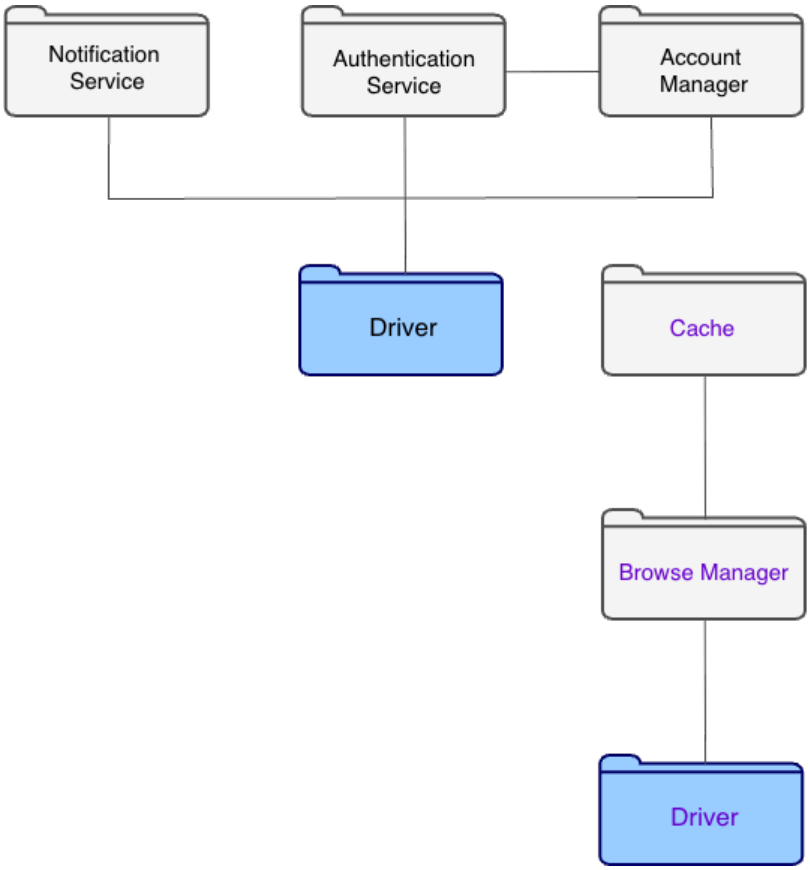


Figure 29: [F2]: Path Browsing and Visualization

This step integrates the *Browse Manager* and the *Cache*. The Driver simulates browsing requests (e.g., view path map, view path details). The test focuses on the cache-first strategy (cache hit / cache miss), and on correct retrieval and formatting of path data. The *Maps Provider* is an external service used for map/geometry retrieval and is not shown in the diagram; in testing it can be simulated to return deterministic map data.

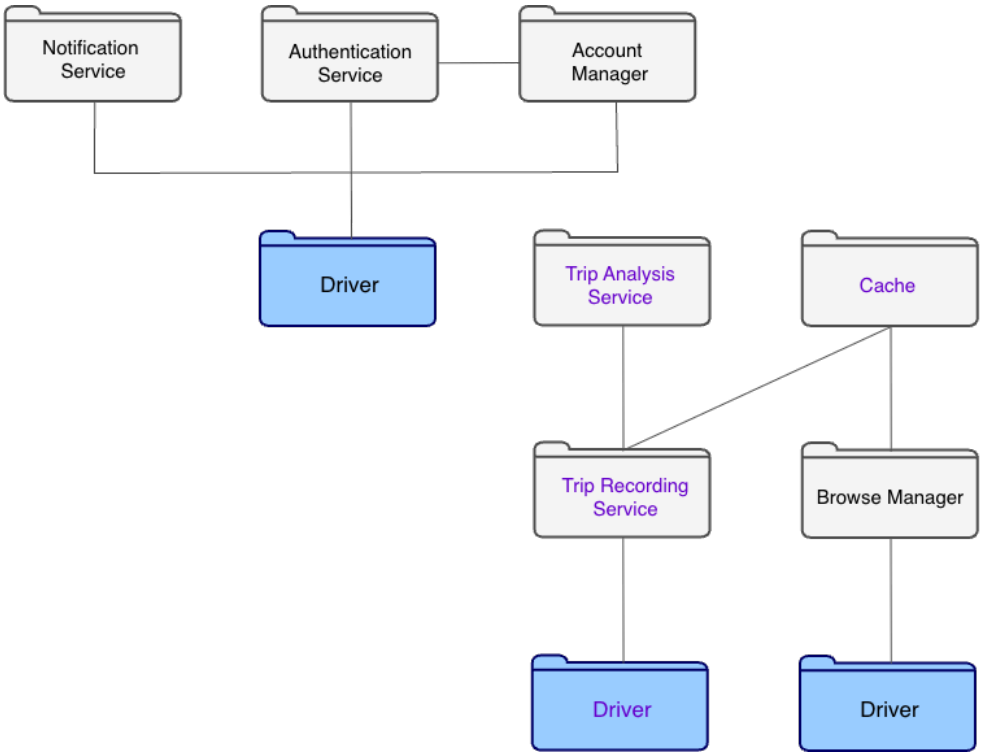


Figure 30: [F3]: Trip Recording

This step introduces the *Trip Recording Service* and integrates it with the already available *Cache*. The Driver simulates the Mobile App streaming trip samples and starting/stopping a recording session. In this phase, the goal is to validate session management, temporary storage of raw samples, and basic consistency of the recording lifecycle. If real-time motion processing is required, *Trip Analysis Service* can be connected at this step to validate the interaction contract (e.g., sending samples for motion event detection); otherwise it can be progressively activated in later steps.

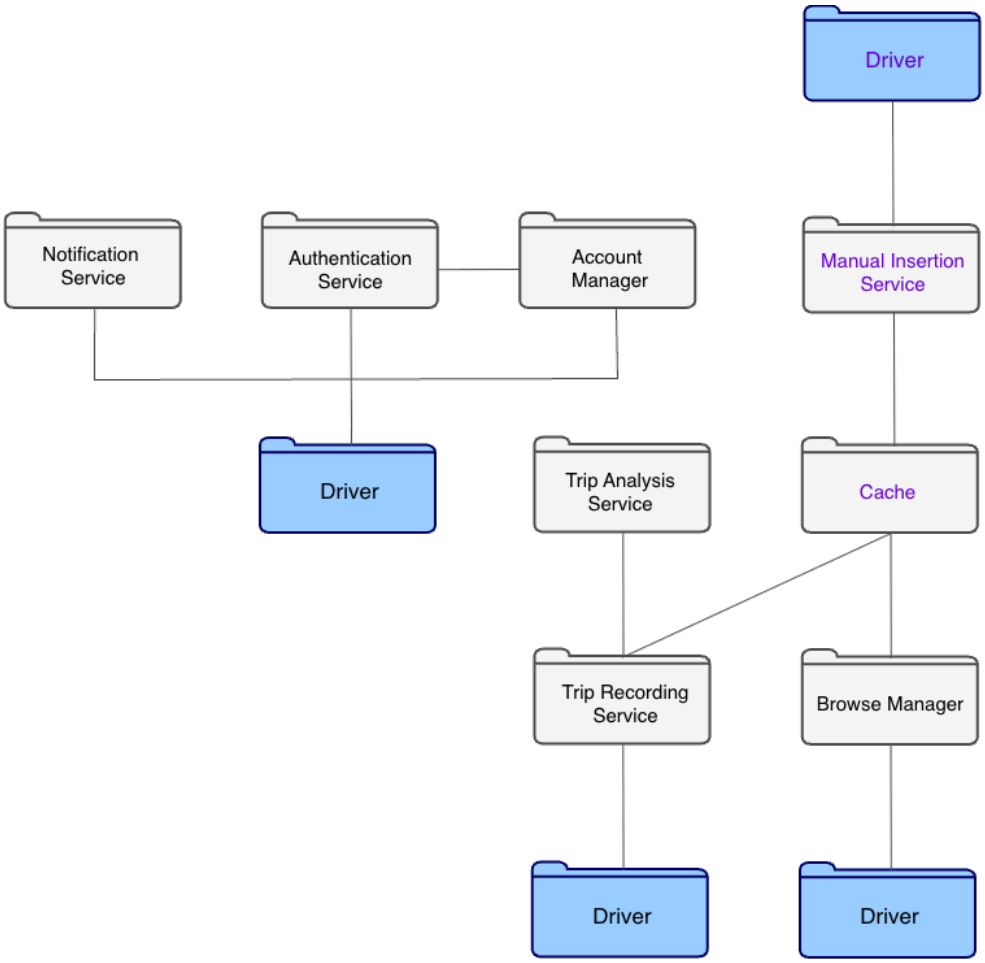


Figure 31: [F4]: Manual Path Insertion

This step adds the *Manual Insertion Service* to allow creation of new paths through map-based interaction. The Driver simulates user actions such as adding points, validating geometry/metadata, and confirming creation. The *Cache* supports temporary drafts, while the final validated path is persisted through the DBMS (assumed available). Any route/segment computation relies on the external *Maps Provider* and is not part of the integration diagram.

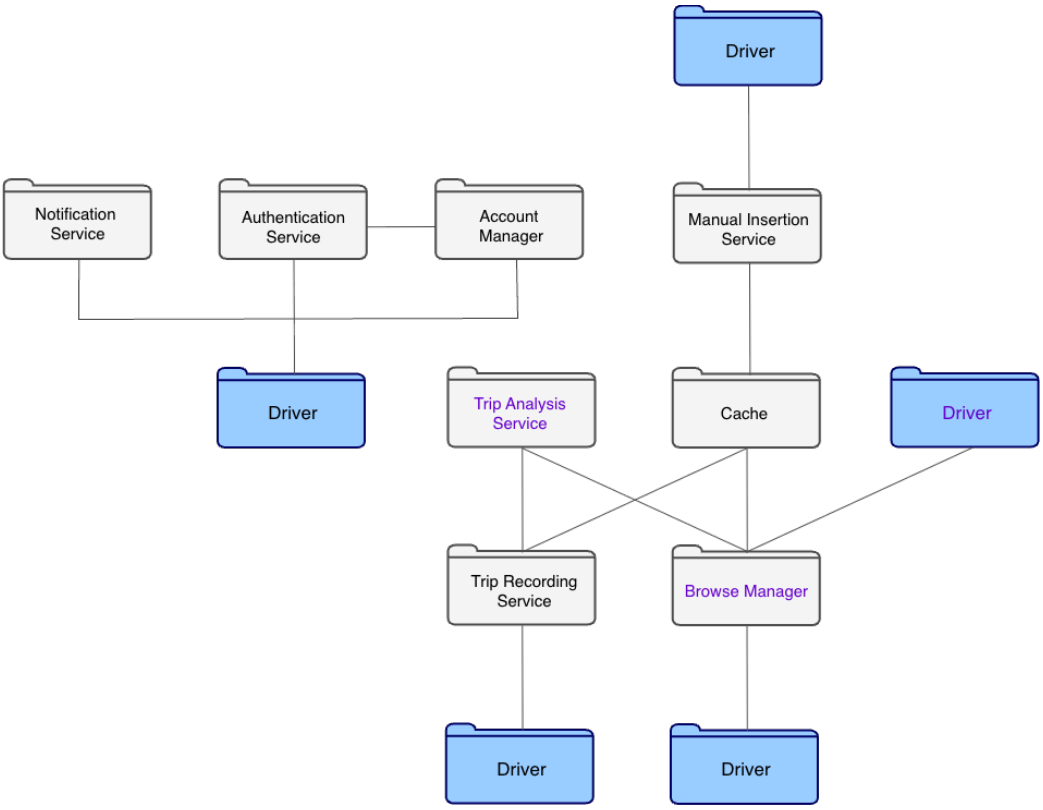


Figure 32: [F5]: Search and Ranked Paths

This step upgrades browsing with the ranking/search logic by integrating the *Trip Analysis Service* in the read pipeline. The Driver triggers search requests; the *Browse Manager* applies filtering, then requests ranking indicators computed from historical data. The test checks:

(i) correct filtering

(ii) correct invocation of the ranking computation

(iii) sorted results returned to the client

Ranked paths are retrieved from the DBMS, while cache can be used to speed up repeated queries.

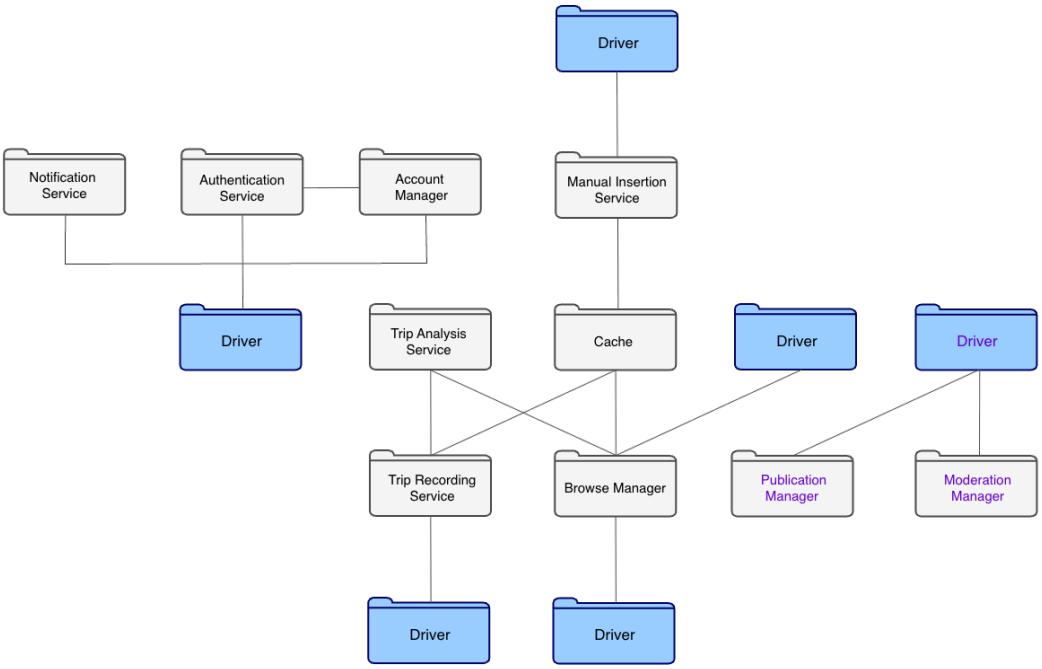


Figure 33: [F6]: Admin Path Management

This step introduces the administrative workflow to view and edit existing paths. The Driver simulates Admin Dashboard requests (list paths, load details, submit updates). The test focuses on retrieval of path data for inspection, and persistence of edits. Map visualization again depends on the external *Maps Provider* and is not shown in the component integration diagram; caching behavior can still be validated for repeated admin queries.

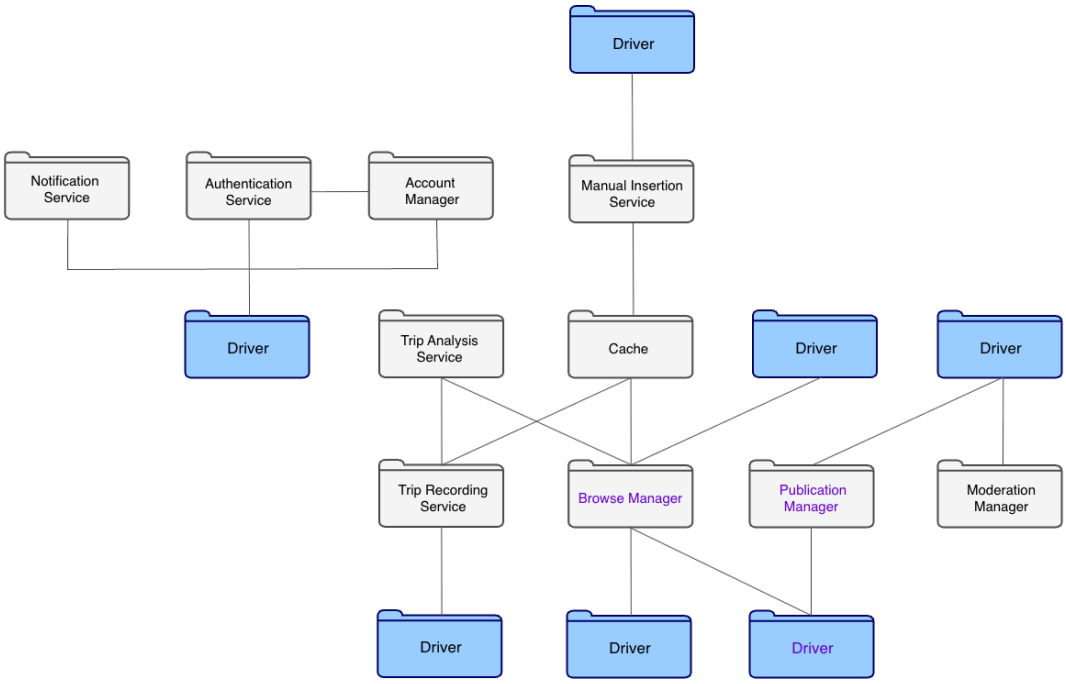
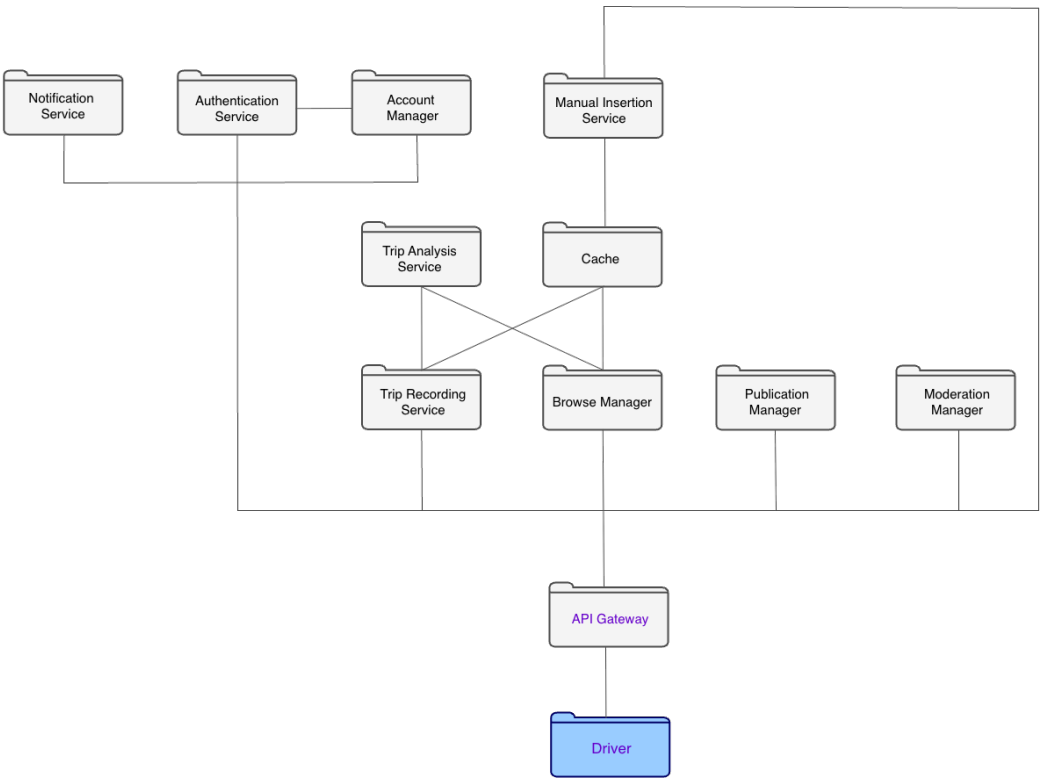


Figure 34: [F7]: Path Publication

The final step integrates the *Publication Manager* to complete the path lifecycle. The

Driver simulates publishing requests and visibility updates. The test verifies eligibility checks, correct state transitions (e.g., `Private` → `Public`), and persistence of the visibility status in the DBMS. At the end of F7, all core services required to browse, create, rank, manage, and publish paths have been incrementally integrated and validated.

During the early incremental steps, dedicated *Drivers* are used to directly invoke backend services and isolate newly integrated components. Once the core backend inter- actions are stable, these Drivers arebreplaced by the API Gateway, which is implemented, unit-tested, and integrated with the other backend components. In this phase, the API Gateway acts as a unified entry point for incoming requests, forwarding them to the appropriate services already validated in the previous testing steps.



5.4 System testing

Once the BBP system is fully integrated, system testing is performed to verify that all functional and non-functional requirements defined in the RASD are satisfied. At this stage, the system is tested as a whole, considering the interaction between all components and services.
System testing is executed in an environment as close as possible to the production one, including communication with external services such as map and weather providers.
The following types of testing are performed.

Functional Testing

Purpose:

- Verify that the system satisfies all functional requirements.
- Check that system behavior is consistent with the use cases described in the RASD.

How:

- The system is used according to the defined use cases.
- Typical scenarios are executed, such as registration, authentication, trip recording, manual path insertion, path validation, route search, publication, and moderation.
- For each scenario, the expected system behavior is verified.

Performance Testing

Purpose:

- Detect bottlenecks affecting response time and resource usage.
- Identify inefficient algorithms.
- Detect possible hardware or network issues.
- Identify optimization opportunities.

How:

- The system is tested under an expected workload.
- Performance metrics such as response time and resource usage are measured.
- Results are compared with acceptable performance levels, especially for critical operations such as route search and map visualization.

Load Testing

Purpose:

- Identify system limits under increasing workload.
- Detect issues such as memory leaks or poor resource management.
- Verify system stability during long execution periods.

How:

- The system is tested with an increasing number of concurrent users and requests.
- Long-running tests are performed to observe system behavior over time.
- Core services such as trip recording, browsing, and data aggregation are monitored.

Stress Testing

Purpose:

- Verify that the system can recover correctly after failures.
- Identify system breaking points.

How:

- The system is stressed by exceeding normal workload levels or by reducing available resources.
- Scenarios include a sudden increase in concurrent users or temporary unavailability of system resources.
- The system is checked to ensure correct recovery and data consistency after failures.

5.5 Additional Specifications on Testing

Testing activities are performed during the entire development of the BBP system and are not limited to the final stages. Unit testing and integration testing are used to validate individual components and their interactions before full system integration.

Inspection activities are also carried out starting from early development phases. Requirements and design documents are reviewed to ensure that goals, requirements, and architectural choices are clear and consistent.

Code inspections are performed to identify errors and inconsistencies. During these activities, developers review code structure and behavior, while fixing detected issues remains

the responsibility of the code authors.

Developers are encouraged to write clear and well-documented code to support testing and maintenance activities. Proper comments and readable code help reduce errors and improve overall system quality.

Finally, developers are expected to produce test cases for critical functionalities such as trip recording, path processing, routing, publication, and administrative operations. Early testing helps detect problems sooner and reduces risks during final system testing.

6 Effort spent

The table below shows the time spent by each team member on the project.

	Introduction	Arch. design	UI design	Req. traceability	Impl. Integ. Test plan
Wojciech Maciej Bukala	5	9	17	3	2
Giacomo Rigo	1	18	1	1	17
Mattia Zonzin	5	13	3	7	10

Table 1: Table of effort spent

7 References

- GitHub was used for version control and collaborative project management.
- The document was written using LaTeX with Overleaf as the online editing environment.
- Sequence diagrams were created using the PlantUML framework integrated in Visual Studio Code.
- Additional diagrams were designed with draw.io.
- Figma was employed for the development of the user interface design.